The Journal of Systems and Software 220 (2025) 112277

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

Retriever: A view-based approach to reverse engineering software architecture models☆

Yves R. Kirschner [a,*], Moritz Gstür [a], Timur Sağlam [a], Sebastian Weber [b], Anne Koziolek [a]

[a] KASTEL – Institute of Information Security and Dependability, Karlsruhe Institute of Technology (KIT), Am Fasanengarten 5, 76131 Karlsruhe, Germany
[b] FZI – Research Center for Information Technology, Haid-und-Neu-Str. 10–14, 76131 Karlsruhe, Germany

ARTICLE INFO

Dataset link: Dataset and Replication Package for the View-Based Retriever Approach To Reverse Engineering Software Architecture Models (Original data)

Keywords:
Automated reverse engineering
Component-based software engineering (CBSE)
Model-driven reverse engineering (MDRE)
Model transformation
Software architecture model
View model

ABSTRACT

Software systems, which have become an integral part of our daily lives, are constantly evolving and growing in complexity. The challenge of understanding and managing these systems has led to a shift towards automated extraction of models from software artifacts. However, extracting architectural models for web service or microservice systems is challenging due to the heterogeneity of formats, languages, and semantics. This challenge arises from the independent deployment and loose coupling of services, as well as the diverse technologies and platforms that comprise the heterogeneous artifacts. This paper addresses this issue with two key ideas: a knowledge representation model for diverse artifacts and a framework for integrating individual views into a unified architectural model. The former involves model-to-model transformations that consider technology-specific relationships and concepts. At the same time, the latter uses model transformation and composition techniques to create a unified model based on an existing metamodel. This work goes beyond the current state of the art by introducing an approach for reconstructing component-based software architectures and combining model-driven reverse engineering processes to capture information across multiple views. As empirical evidence of its applicability and accuracy, we report on its successful application in real projects of different sizes. Our evaluation shows scalability in generating architectural models and can reconstruct the components of a software system and their interactions with F_1 values up to 0.94 for structural properties and up to 0.88 for behavioral properties.

1. Introduction

In today's digital age, software systems have entered every aspect of our lives, from smartphone applications to complex infrastructures that power the internet. These systems are constantly evolving, growing larger, more complex, and more mission-critical every day (Murer et al., 2011). Understanding and managing these software systems has become a major challenge for academia and industry (Canfora and Di Penta, 2007). To understand a software system, one must first uncover its architecture, which describes its structure, its components, and their interconnections. Architecture is the key that holds the system together and influences its performance, scalability, maintainability, and security (Kazman et al., 1994). As systems become more complex, it becomes increasingly difficult to create a comprehensive and accurate representation of the architecture description. Historically, creating and maintaining architectural description models has been a largely manual and error-prone process (Canfora and Di Penta, 2007), in which software architects and engineers painstakingly document system architectures, often in the form of diagrams and textual descriptions. This manual approach not only consumes valuable time and resources, but is also prone to human error and inconsistency. In response to these challenges, interest in automatically extracting models from software systems has increased recently (Lungu, 2008). This involves using automated tools to extract models from software artifacts such as source code, documentation, and configuration files. Automatic extraction of models from software systems has several advantages over manual modeling. It is much faster, less prone to error, and makes more fine-grained modeling possible for systems that are too complex for manual modeling (Selfridge et al., 1993).

The automatic extraction of architectural models from software artifacts for web service or microservice systems is challenging due to the heterogeneity of formats, languages, and semantics (Chu et al., 2002). On the one hand, the independent deployment and loose coupling

☆ Editor: Raffaela Mirandola.

* Corresponding author.
E-mail addresses: yves.kirschner@kit.edu (Y.R. Kirschner), moritz.gstuer@student.kit.edu (M. Gstür), timur.saglam@kit.edu (T. Sağlam), sebastian.weber@fzi.de (S. Weber), anne.koziolek@kit.edu (A. Koziolek).

https://doi.org/10.1016/j.jss.2024.112277
Received 29 December 2023; Received in revised form 24 September 2024; Accepted 3 November 2024
Available online 9 November 2024
0164-1212/© 2024 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

of services in such systems make it difficult to extract a consistent architectural model. On the other hand, the heterogeneous software artifacts are composed of different technologies and platforms (Bogner et al., 2019), which also requires an understanding of the interaction of different components. Different software artifacts, such as source code and configuration files, can exist in different formats and programming languages. This diversity makes it difficult to develop automated tools to extract models from all types of software artifacts. In addition, the different syntax and semantics of the various formats and languages further complicate the development of a unified extraction approach. For example, the semantics of source code may differ significantly from those of configuration files. This can make it difficult to automatically infer relationships between different concerns of a software system. Finally, software artifacts often contain incomplete information (Pfeiffer, 2020), such as missing deployment in the source code. This requires additional information to build a complete system model. In addition, software systems are characterized by complex relationships between components and configurations that require advanced analysis techniques to be accurately captured and represented in a model. Although the current state of the art supports a wide range of programming languages, support for heterogeneous systems consisting of multilingual source code or other types of artifacts, such as configuration files, is still being explored (Hasselbring, 2002; Ali et al., 2018).

The research questions are derived from this problem definition as formulated objectives. The basic research questions to be answered by the implementation of the approach are as follows:

RQ1 What is an effective method for reverse engineering software architecture from disparate artifacts across technologies?

RQ2 How can different perspectives of a software system be integrated into a unified model that addresses different concerns?

Based on these research questions, we present a **R**everse **E**ngineering **T**echnique with **R**efinement **I**ntegration of **E**xtracted **V**iews with **E**lements and **R**elationships (*Retriever*) to answer them. Our *Retriever* approach is based on two key ideas, which we present in this paper, that address the concerns and perspectives of different views of a software system. The first idea we present is to develop a knowledge representation model for software architecture views that can cover the different technologies, formats, and languages used in the different artifacts. Artifacts in this context can take various forms, such as Java or JavaScript source code files, YAML or JSON configuration files, or Maven or Gradle files used for build automation. The idea is to model this knowledge representation as model-to-model transformations to reconstruct views from the system artifacts, considering technology-specific relationships and concepts. This knowledge representation can cover the different types of views and their concerns for that view. The second idea we present is a framework for integrating individual views into a unified, overarching architectural model. This framework uses model transformation and composition techniques to map concepts between view-specific models and incorporate them into a unified model based on an existing metamodel. To this end, we have developed a view composition algorithm that harmonizes the different views of a software system and produces a unified software architecture model. The objective of this model is to predict quality, which can extend beyond the scope of mere architectural documentation. Therefore, the presented approach can be considered a predecessor to quality prediction since it can automate the creation of models suitable for quality prediction that can be used with specialized tools.

Based on these two ideas, we have formalized and developed our *Retriever* approach that can handle different formats and languages and translate them into a common representation for architectural modeling. Based on this formalization and development, we make the following contributions that go beyond the state of the art:

C1 A technique for transforming reverse-engineered models into more accurate representations that approximate the component-based software system architecture.

C2 A technique that combines model-driven reverse engineering processes to capture information while processing multiple views and resolving overlaps between them.

The evaluation was conducted on 16 diverse and representative open-source projects chosen for their use of key technologies and popularity. These projects varied in size and complexity, allowing for a thorough examination of the architecture model generation and its scalability. The findings indicated that execution time was correlated with the size of the input code base. The ratio of source files to components in the final model confirmed that the approach successfully abstracts from the source code to produce valid architectural models. A comprehensive comparative analysis of automatically generated and manually created models for seven projects revealed that the approach effectively recovers and analyzes the majority of components and their interrelationships. The approach achieved F_1 values of up to 0.94 for structural properties and 0.88 for behavioral properties.

In a previous study (Kirschner et al., 2023b), we investigated the potential of trace links to connect vulnerabilities in code to elements in architectural models. This paper presents a method for reverse engineering with different technologies and discusses integrating the different views that result in an architectural model. Furthermore, we consider other behavioral aspects beyond merely identifying software vulnerabilities.

The paper is organized as follows: Section 2 reviews the background literature and key concepts. Section 3 describes the research methods, algorithms, and techniques used. Section 4 presents the open-source implementation of our *Retriever* approach. Section 5 discusses the evaluation process and results. Section 6 compares our work with related literature. Finally, Section 7 summarizes the results and outlines future research directions.

2. Foundations

This section presents the foundations of this work, focusing on model-driven software engineering, component-based software engineering, architecture description languages, and software architecture reverse engineering. It emphasizes the intersections between our reverse engineering work and these key concepts.

2.1. Model-driven engineering

Model-driven engineering (MDE) emphasizes the use of models as fundamental elements throughout a development process, so that they represent the system at different levels of abstraction (Kent, 2002). Models represent natural or artificial originals for a specific purpose (pragmatism) and include only attributes that serve that purpose (reduction) (Stachowiak, 1973). A metamodel, in contrast, is a model of models, describing their properties and elements (Object Management Group, 2017). In model-driven software development (MDSD), models and metamodels are employed alongside code during software development, emphasizing modeling and *model transformation*. This entails refining abstract models and translating them into technical implementations (Völter et al., 2013). The Eclipse Modeling Framework (EMF) is the most widely used framework for model-driven development. It provides a meta-metamodel called *Ecore*, which enables the development of domain-specific metamodels (Steinberg et al., 2008).

Model transformations enable the transformation of one model into another of a different type or level of abstraction. They can be categorized into two types. Endogenous transformations involve source and target models of the same metamodel and are typically used for refinement or abstraction. Conversely, exogenous transformations involve source and target models of different metamodels and are often used for platform-specific implementations (Mens and Van Gorp, 2006). Typical transformation languages are QVT (Kurtev, 2008) or ATL (Jouault et al., 2008). Besides these specialized transformation languages, general purpose languages (GPLs) such as Java or Xtend (Bettini, 2016) can

also be used for model transformations (Hebig et al., 2018). In contrast to GPLs, which are employed to develop software in a wide variety of application domains, domain-specific languages (DSLs) are tailored for a specific domain GPLs offer a wider range of features and high flexibility, making them suitable for various tasks (Götz et al., 2021).

2.2. View-based modeling

Stakeholders typically focus on a particular subset of the system, and their ability to hide the inherent complexity of the system allows them to focus solely on their concerns (Wittler et al., 2023). In addition, certain stakeholders may have limited access to the model or be prevented from modifying parts of it. View-based development facilitates the separation of concerns and provides stakeholders with only the information necessary for their individual concerns (Cicchetti et al., 2019). Views are part of the larger domain of architectural description languages and are used to effectively manage the complexity of large software systems. These techniques divide systems into multiple views or perspectives, each of which addresses different aspects of the system. Views can represent structure, behavior, or usage, while perspectives offer different interpretations of these views. Model transformations allow defining and updating views, enabling tasks such as code generation, behavior simulation, or system property analysis. The *4+1 View Model* introduced by Philippe Kruchten (Kruchten, 1995) and the rational unified process (RUP) (Kruchten, 2004) are among the most recognized methods in this area.

ISO/IEC 42010 is a standard for architectural description practices, outlining the relationship between models and views in representing system architecture. It mandates model-view consistency to ensure coherence across different representations, with a view focusing on specific concerns and a model as an abstract representation depicting architectural elements, relationships, and properties (ISO Central Secretary, 2022). The standard distinguishes between *projective* architectural views and *synthetic* architectural views. Projective views are derived from an underlying model, whereas synthetic views and their associated relationships constitute the underlying model. *Projective Views* provide a means of establishing perspectives on the underlying model, allowing stakeholders to examine the model from different *viewpoints* (Atkinson et al., 2010). A view is projected from a model by applying a particular *view type* to the model (Goldschmidt et al., 2012). As a result, a view type acts as a metamodel for the view (Burger, 2013), outlining the elements and relationships that it can contain. Therefore, views abstract from the totality of the modeled information and its complexity, making them an optimal solution for managing stakeholder interactions with the model (Cicchetti et al., 2019).

2.3. Component-based software engineering

Component-based software engineering (CBSE) is a software engineering paradigm that advocates the construction of software systems using reusable, interchangeable elements called components. This methodology promotes modularity, reusability, and separation of concerns, providing an efficient strategy for managing the complexity of large software systems. CBSE can be implemented at various levels of granularity, ranging from small components within a software system to large subsystems that make up an entire application. It emphasizes the development of reusable components that encapsulate different functionalities and interact through well-defined interfaces (Szyperski et al., 2002). Components are independent software units that are loosely coupled, which promotes flexibility and adaptability in system design (Gruhn and Striemer, 2018).

2.4. Architecture description languages

Architecture description languages (ADLs) are specialized modeling languages to describe the structure and behavior of a software system and the non-software entities with which it interacts. A system is represented as a collection of software components, their interconnections, and their significant behavioral interactions (ISO Central Secretary, 2022).

2.4.1. Unified modeling language

The Unified Modeling Language (UML) is an architecture description language (ADL) that serves as a visual language for designing, documenting, constructing, and presenting software and other systems (Object Management Group, 2017). UML model elements, the basic units of a model, are used to represent the architectural design of a system in diagrammatic form. A named element, a model element that can be assigned a name, supports the use of a string expression for naming, allowing template parameters to be included in model element names. UML relationships represent the connections between structural, behavioral, or grouping elements. Often referred to as a link, a UML relationship describes how multiple elements can interact during system execution. Artifacts in UML models represent a piece of information that is used or produced by the operation of a system. Artifacts can take the form of model files, source files, or development deliverables (Object Management Group, 2017). The UML can be effectively represented using the PlantUML tool (Roques, 2023). This open-source tool is specifically designed to facilitate the creation of UML diagrams and supports a wide range of diagram types.

2.4.2. Palladio component model

The Palladio Component Model (PCM) is an ADL designed to model, analyze, and predict the quality attributes of software systems. It encapsulates both the static and dynamic structure of component-based software architectures, enabling software architects and engineers to reason about system properties and identify optimization opportunities during the design phase (Becker et al., 2009). The PCM is supported by the open-source Palladio simulator, which enables a range of simulations and quality predictions. The Palladio approach is model-centric, with structural, behavioral, usage, and deployment views serving as Palladio's first-class entities (Reussner et al., 2016). These views represent the primary elements of information about a software system.

The PCM is based on the EMF and is the subject of the technical report (Reussner et al., 2011) that contains a detailed description of the PCM metamodel. The core components of the PCM metamodel relevant to the present work are presented below. The structural view focuses on the static architecture, capturing components, interfaces, and assembly connectors. The behavioral view focuses on the dynamic aspects, capturing interactions and message flows during runtime. The usage view focuses on the use of the system by users or external systems, capturing usage scenarios and profiles. The deployment view focuses on the allocation of components to resource containers and the distribution of the system across the hardware infrastructure. By fusing and analyzing information from these views, software architects gain a comprehensive understanding of system behavior and properties, helping them make informed design decisions. Key features and concepts of the PCM include components, assembly connectors, resource containers and requirements, and ServiceEffectSpecifications (SEFFs). Components serve as the basic building blocks of the system, with defined interfaces for interaction. Assembly connectors define the communication patterns between components. Resource containers represent the environments in which components operate, providing CPU, memory, and storage resources. Resource requirements detail the resource needs of components, while SEFFs define the impact of component services on resources (Becker et al., 2007).

Our approach uses the PCM as the metamodel for constructing the software system models. This metamodel is used because it ensures that the models adhere to rigorous semantic criteria and accurately encapsulate the structure and behavior of the system. However, while the PCM serves as one of the metamodels that define the structure and semantics of our software system models, the underlying concepts of our approach remain independent of this metamodel.

2.5. Software architecture reverse engineering

Reverse engineering of software architecture is a process that attempts to reveal the underlying structure of a software system, including its components and how they are interconnected. This process is particularly useful for understanding systems that have evolved or that lack formal documentation (Kazman et al., 1998). However, manual reverse engineering of a software architecture can be a challenging and labor-intensive task, especially for large and complex systems. Automating this process typically involves code analysis to identify components, interfaces, dependencies, and other architectural elements (Canfora et al., 2011).

Although automated reverse engineering can be helpful, the resulting architectural model is not always complete or accurate, and may need to be manually reviewed and refined. Reverse engineering is always ambiguous because the correct results depend heavily on the chosen mapping between architecture and source code, which is often implicit and exists only in the minds of software developers or architects. Furthermore, the quality of the extracted architecture model depends on the quality of the techniques used to extract it. Since an architecture contains aspects that are not present in the source code, reverse engineering must use more than just the source code as input to achieve the best possible results (Reussner et al., 2016).

2.5.1. Model-driven reverse engineering

Model-driven reverse engineering (MDRE) uses the principles of MDE to analyze and understand existing software systems. It uses models to represent the structure, behavior, and other aspects of the system, providing a high-level abstraction that facilitates analysis, understanding, and modification. Compared to traditional reverse engineering methods, MDRE offers improved scalability, maintainability, and reusability of results (Rugaber and Stirewalt, 2004). MDRE enables reverse engineers to focus on understanding the system without being overwhelmed by the complexity of the code (Favre, 2005). MDRE aims to transform disparate software development artifacts into unified models. The MDRE process involves two primary steps (Raibulet et al., 2017): (1) transforming the software system into a set of models of software development artifacts without omitting necessary information, and (2) generating the required output models through model transformations.

2.5.2. Metrics for evaluating reverse-engineered models

The evaluation of reverse-engineered models against a gold standard model provides insights into the accuracy and effectiveness of various reverse engineering techniques and tools. The metrics commonly used for this purpose include precision, recall, and F_1 score. The precision, recall, and F_1 metrics facilitate a comparison between the model's predictions and a gold standard, providing an objective means to compare the performance of reverse engineering approaches (Van Rijsbergen and Van Rijsbergen, 1979). Precision (also called positive predictive value) is the proportion of relevant instances among the instances found, i. e. $p = \frac{TP}{TP+FP}$. In the context of reverse engineering, it can be interpreted as the ratio of correctly reverse-engineered elements to all elements that the model identified as belonging to a given class. Recall (also called sensitivity) is the proportion of relevant instances retrieved, i. e. $r = \frac{TP}{TP+FN}$. In this context, it can be interpreted as the ratio of correctly reverse-engineered elements to all elements that truly belong to a given class according to the gold standard. The F_1 score is the harmonic mean of precision and recall, i. e. $F_1 = 2 \times \frac{p \times r}{p+r}$.

3. Approach

We now present our main contribution, a **R**everse **E**ngineering **T**echnique with **R**efinement **I**ntegration of **E**xtracted **V**iews with **E**lements and **R**elationships (*Retriever*). This approach enables the reverse engineering of software architecture models from heterogeneous artifacts, thus allowing to reverse engineer architectural models to predict quality, surpassing mere architectural documentation. In this section, we present our *Retriever* approach conceptually, while the technical implementation is described in the following Section 4. The presentation of our approach consists of several subsections, including terminology, extracting views from artifacts, and composing and refining views. The reverse engineering approach uses existing software artifacts to extract structural and behavioral views using rule-based processes. These views are then refined to ensure completeness and consistency, with model-driven composition connecting the various elements. The process concludes with a transformation that produces output views tailored to user needs. These views provide a comprehensive understanding of software systems and support model-based analysis and quality prediction.

3.1. Reverse engineering terminology

As part of our approach, we present a new terminology that serves as the conceptual framework of our approach, providing a nuanced understanding of its critical elements. A high-level view of the terminology concepts and their relationships is shown in Fig. 1. Our terminology is based on the terms *Metamodel*, *Model*, *ViewType*, and *View* of the UML (Object Management Group, 2017) and the terms *System*, *Artifact*, *ModelElement*, *NamedElement*, *Relationship* of the ISO/IEC 42010 (ISO Central Secretary, 2022) already introduced in Section 2. For our terminology, we have related these terms to each other and extended the existing terminology to include the terms *TraceLink*, *ExtractionRule*, *RefinementRule*, and *FinalizationRule*. In this context, the composition rule is subsumed within the refinement rule, as both are part of the same coherent process, as described in Algorithm 3.

A *TraceLink* is an association that connects model elements shown in a view (the target) to artifacts used or produced by a software development process or by the deployment and operation of a system (the source). An *ExtractionRule* defines how a particular concept is applied in an artifact and how it affects the system. These extraction rules are implemented as model-to-model transformations that create instances of views based on the artifacts that represent parts of the system. A *RefinementRule*, another type of model-to-model transformation, first checks whether a model element in the view meets certain criteria. If the criteria are met, the view remains the same. If not, the transformation generates new model elements, called implications, that must be incorporated into the view through this transformation to satisfy the criteria. A *FinalizationRule* describes how the model elements shown in a view are transformed into a new view. This transformation, which occurs during the final transformation process, acts like a filter, selecting elements from the original view and merging them into a new target view based on specific conditions.

The refinement and finalization rules do not depend on the specific application. However, the extraction rules must be defined manually and are particular to the application and technology used. It is conceivable that technology experts or system architects could devise rules for the automated reverse engineering of software architectures. They could leverage their understanding of specific technologies and architectural objectives to implement corresponding rules. These rules can be implemented using model-to-model transformations, which are carried out using different programming languages. These rules define the assignment of software elements, such as classes and methods, to architecture components and relationships. These rules are encapsulated in a descriptor format that contains loops for iterating over elements of software development artifacts, conditions for logical tests,

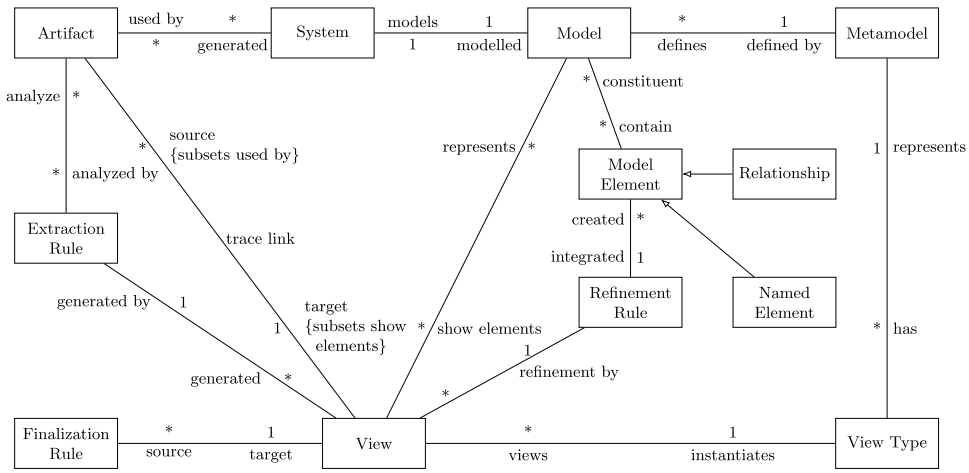


Fig. 1. View-based reverse engineering terminology used in this approach.

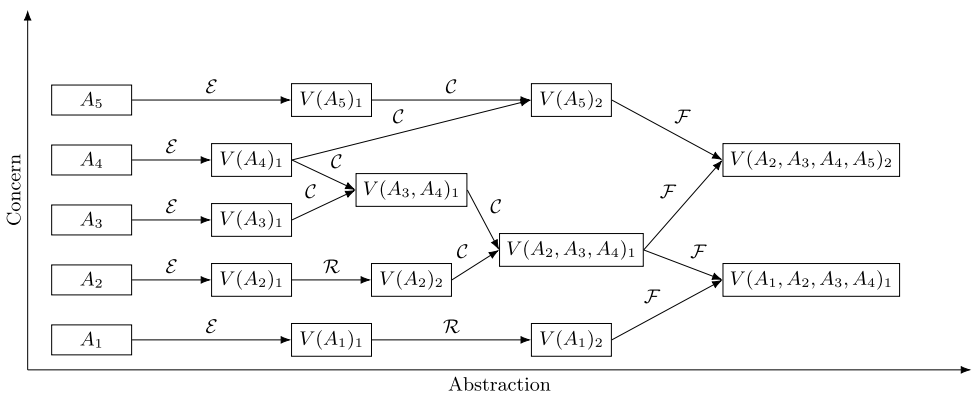


Fig. 2. Two-dimensional arrangement of views, where the vertical dimension represents the concerns and the horizontal dimension indicates the models' abstraction level. Artifacts A are extracted $\mathcal{E}$ into views $V(A)$, then refinement $\mathcal{R}$ enhances these views, and combination $\mathcal{C}$ integrates different views. Finally, finalization $\mathcal{F}$ merges elements from views into a new target view.

and recognitions for mapping code structures to architectural elements. In this context, programming languages are considered models because they abstract and represent complex ideas and arithmetic operations in a structured and standardized way.

The separation of concerns principle helps control complexity in developing large software systems. Applying this principle in model-based reverse engineering results in more than one system model being developed from different perspectives. Therefore, our approach to model-based reverse engineering can also be viewed as a series of refinement steps: More abstract models are transformed into more detailed models closer to the actual system. The critical requirement of this refinement process is that the transformations preserve semantics. These models can be refined independently along a single path. However, these models must be integrated at some point to obtain a complete system. The model composition can be considered the transformation of at least two input models into a composite model. Refinement and composition transformations must be semantically preserving, since the resulting models represent the same system as the input models. This refinement and composition approach is based on the transformation scenarios in software and data engineering described by Kurtev (2008), which we transfer to the scenarios of model-based reverse engineering.

Fig. 2 shows an example of the arrangement of models in a two-dimensional space. The vertical dimension shows that views can be organized according to the problems they solve, called concerns. At each stage of our model-based reverse engineering approach, models can deal with multiple concerns. The horizontal dimension indicates the level of abstraction of the models. On the left are the artifacts A from which the more concrete views $V(A)$ of the system are transformed. An extraction $\mathcal{E}$ creates an instance of views based on the artifacts that represent parts of the system. In a refinement $\mathcal{R}$, the information from the artifact is transformed into a more concrete view. This view can then be refined or combined $\mathcal{C}$ with information from the artifact or other views. A transformation $\mathcal{F}$ selects items from all views and merges them into a new target view based on the selected view. Our idea here is that only $\mathcal{E}$ is technology or project-specific. $\mathcal{C}$, $\mathcal{R}$ and $\mathcal{F}$ are generic for component-based systems. Therefore, the approach allows new combinations of the extracted views, where only E needs to be adapted or extended.

3.2. High-level problem description

This Section 3 describes our approach to addressing the challenges of extracting, refining, composing, and finalizing software architecture models. We present a systematic approach that facilitates interpreting the complex relationships within views and streamlines this integration into a unified architectural model.

The first step, outlined in Section 3.4, involves extracting components, along with their respective interfaces and deployment configurations, from heterogeneous software artifacts. This process is critical because it is the basis for generating an accurate software system architecture model. Advanced parsing and analysis techniques extract structural and behavioral views from various artifacts, including source code, configuration files, and deployment descriptors. The extraction

process must be performed correctly to capture the software architecture's essential elements accurately. This ensures that subsequent refinement and composition stages are based on views that accurately reflect the actual structure and behavior of the system. For illustrative purposes, an example of this step is given in Section 3.7.2.

Following the extraction process described in Section 3.5, the next step is to refine and compose the extracted views. The refinement processes are designed to increase the granularity and precision of the architectural views by resolving inconsistencies and incorporating rich connections between views. Subsequently, composition techniques are applied to integrate the refined views into a unified architectural representation. This integration is critical for comprehensively understanding the system's structure and behavior. An example is provided in Section 3.7.3 for illustrative purposes.

The final stage of our approach is creating a comprehensive architectural representation, as discussed in Section 3.6. The synthesis of the extracted and refined views provides a unified model that facilitates a thorough understanding of the system's structure and behavior. This representation is a central tool for stakeholders, providing a comprehensive and navigable visualization of the system's architectural intricacies. It enables various analytical pursuits, including assessments of performance, scalability, and maintainability, and thus contributes to the strategic planning and evolution of the software system. An illustrative example is given in Section 3.7.4.

Our approach employs view-based engineering principles and integration and model transformation techniques. It provides a solution that can accommodate the complexity of modern software systems, which often exhibit heterogeneity in their technological composition. The following sections present the methods and algorithms used to implement our approach.

3.3. Reverse engineering process

Our reverse engineering process described transforms artifacts, each with specific concerns, into views that comprehensively represent a system using a structured approach with three main inputs: *Extraction Rules* E, *Refinement Rules* R, and *Finalization Rules* F. The process commences with identifying artifacts pertinent to the software system, such as source code or configuration files. The artifacts are identified and analyzed using so-called *Finders*, which can utilize third-party parsers to transform the artifacts into initial models for further extraction. This configuration permits the integration of additional *Finders* as required. These *Finders* are used to identify and analyze artifacts, while the *Extraction Rules* transform the models generated by the *Finders* into initial extracted views for the next refinement steps.

Subsequently, in Section 3.4, extraction rules are applied to these models to generate abstract views of the software system's architecture. These views are constructed through model-to-model transformations, resulting in representations that highlight components, dependencies, and their distribution. Subsequently, in Section 3.5, each view is refined by verifying that the elements align with predefined criteria and, if necessary, adjusting or adding new elements to meet these criteria. In conclusion, in Section 3.6 the process culminates in integrating all view elements into a unified model that reflects the entire architecture. This integration provides a higher level of abstraction, thereby enabling more effective model-driven analysis.

3.4. Extract views from artifacts

The objective of the process is to extract views from software artifacts, to automate the modeling of software architecture. Initially, structural view types are generated, representing the system structure, including components, interfaces, and connections such as dependencies and distribution relationships across nodes. Subsequently, behavioral view types are developed to capture the interactions among these components, detailing the externally visible actions and dynamic behaviors, including the relationships between provided and required interfaces.

The extraction rules for module decomposition are oriented towards the static structure, with the objective of mapping code to architectural components and identifying interactions based on static code analysis. In contrast, the usage extraction rules emphasize dynamic interactions, such as method calls and data flows, thereby demonstrating to which extent rule-based methods can be employed to capture the intricacies of software architecture. Moreover, trace links are established to develop behavioral views, which detail the mapping of classes, interfaces, and methods to architectural elements such as components and signatures. These links are essential for establishing the mapping of code methods to architectural signatures while creating these elements.

3.4.1. Extract structural views

This step is concerned with the automated extraction of structural views from software artifacts, such as source code and configuration files, to create a model that illustrates the system's component structure, dependencies, and component distribution across nodes. This process involves transforming textual descriptions into analyzable views using model-driven development tools, thereby enabling higher-level analyses. Model transformations apply mapping rules that articulate system requirements or design specifics, thereby capturing the influence of technology-specific implementations on the architecture. The aforementioned rules may be either technology-specific or project-specific, thereby enhancing flexibility and reusability across different projects.

How concepts applied to an artifact influence the resulting system model is defined by extraction rules. In essence, they do so by creating views that represent different system components. These rules are tailored to specific view types, such as layered or decomposition views, to ensure a comprehensive and consistent representation of relevant architectural aspects. Moreover, the extraction process considers the distinctive characteristics of the technology employed, resulting in the creation of rules capable of accurately interpreting software artifacts according to their architectural roles.

Rules for module decomposition may map code to architectural components based on the static aspects of the system. Interface extraction rules identify and map methods and classes that fulfill specific architectural roles to corresponding elements, such as interfaces and signatures. To illustrate, in a Spring framework, a class may be designated as a component if it is annotated in a specific manner. However, other classes may serve as data transfer objects or utility classes, necessitating the development of nuanced rules to accurately capture the architectural details.

The components of a system are not merely clusters of classes; they are also defined by their functionality, roles, relationships, provided interfaces, and their distribution across the system. These definitions illustrate the flexibility and depth of the extraction processes. Moreover, the rules facilitate the identification of specific architectural styles and patterns, such as model-view-controller or microservices, which are often made explicit through code annotations or naming conventions in configuration files. While some architectural styles are readily discernible due to explicit coding conventions, more abstract patterns present significant detection challenges due to the lack of clear code markers or reliance on developer intent.

A thorough understanding of the software system's architectural framework and the intricacies of its technologies is essential to defining these extraction rules. These rules are formulated as model-to-model transformations that require synthesizing software engineering knowledge and domain-specific expertise. This complexity stems from the need to accurately map various software artifacts to architectural elements, considering other technologies' variable semantics and structures. Defining the relationship between software artifacts and architectural elements contributes to the architectural understanding and documentation of the system by creating these rules. The extraction

rules are designed to facilitate adaptation across systems. However, the extent to which they can be considered generalizable depends on the degree to which the technologies used in these systems share commonalities. For example, rules established for Java-based systems may not directly apply without modifying systems developed primarily in other languages or platforms.

The extraction rules serve as a meta-metamodel that standardizes the extraction and transformation across various software artifacts, thereby reducing the complexity and effort required for maintaining technology-specific rules. This meta-metamodel framework facilitates reverse engineering across different programming environments by standardizing common structural elements, such as class definitions and method calls.

Finally, trace links in this approach facilitate mapping code elements, such as classes, interfaces, or methods, to architectural elements, such as components, interfaces, or signatures. Trace links serve as a crucial means of mapping, although they are not a prerequisite for subsequent stages, such as composition, refinement, or finalization of the model.

Algorithm 1 describes our method for rule-based extraction of views from artifacts using a set of predefined extraction rules. This process involves evaluating each artifact against all rules to identify applicable transformations. Specifically, for each pair of artifacts a from the set of artifacts A and a rule r from the set of extraction rules E, the predicate $r(a)$ determines whether the rule r applies to the artifact a. If so, the rule is executed to transform the artifact into a view $v \in V$, which is then recorded as a trace link next to the original artifact in the set L. This resulting set L containing all such trace links with their corresponding views is the output of the extraction function.

Algorithm 1 The rule-based extraction of structural views from given artifacts.

```
function ExtractStructuralViews(A, E)
    L ← { }
    for all a ∈ A, r ∈ E do
        if r(a) then
            L ← L ∪ (ℰ_r(a), a)
        end if
    end for
    return L
end function
```

3.4.2. Extract behavioral views

Applying quality prediction approaches to reverse engineering models requires a representation of component behavior. We transform the behavior descriptions available in the code model into abstract behavior at the component level. For example, all internal component actions are reduced to individual nodes in the behavior model. Any control flow that does not impact other components (no calls to other components) is abstracted.

Algorithm 2 shows our implementation for rule-based extraction of behavioral views for the specified artifacts, with the associated links to the previously extracted structural views. After the trace links are created, the transformation rules are applied to generate the behavioral views atomically. The extraction rules in this step create a view of the system behavior. For the Cartesian product of all used trace links L with all existing rules E, it is first checked whether an extraction rule $e \in E$ applies to a trace link $(v_s, a) \in L$. Here $e(v_s, a)$ is a predicate that indicates whether the rule r applies to the structural view and its associated artifact a.

If so, this extraction process is applied, which takes a trace link as input and is based on an extraction definition e that specifies how to transform the structural view and artifact into a behavioral view $\mathcal{E}_e(v_s, a) \rightarrow V$. These extraction rules perform a control flow transformation of the source code for each class or interface method corresponding to an interface signature. We use Krogmann's approach to extract behavioral views of components from source code by identifying methods that interact with external entities and transforming their control flows into actions in the behavioral model. This includes transforming control structures and consolidating sequential internal actions into abstract actions suitable for high-level architectural analysis. This transformation examines each statement and, if necessary, generates an action in the behavioral view. The most important statements in this context are calls from one component to another, representing data flow and control between components. Component-internal calls, that is, method calls that call a method of the same class or a method of a class within the same component, are also part of the control flow transformation. Other control flow elements, such as conditional statements or branching and looping or iteration, are also represented in the behavioral view. The statements are grouped for better abstraction from the source code. Each extracted behavioral view is stored, along with the views from the given trace link, in the set v_b and returned at the end of the function.

Algorithm 2 Rule-based extraction of behavioral views from given trace links between structural views and artifacts.

```
function ExtractBehavioralViews(L, E)
    V_b ← { }
    for all (v_s, a) ∈ L, e ∈ E do
        V_b ← V_b ∪ v_s
        if e(v_s, a) then
            V_b ← V_b ∪ ℰ_e(v_s, a)
        end if
    end for
    return V_b
end function
```

The example in Fig. 3 shows how the Java code from Fig. 3(a) can be transformed into a state chart in Fig. 3(b). The first two variable assignments have been combined into an internal action. The branch has been taken and shows the external method call in another component and the internal method call within the component.

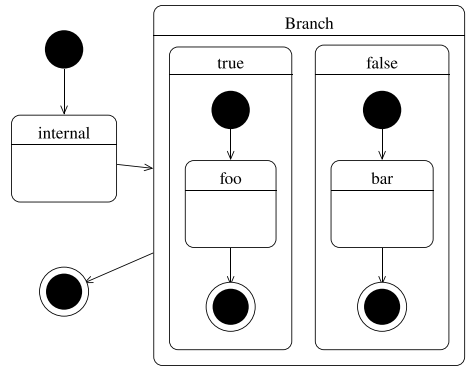
In addition to extracting behavioral views through static code analysis, our approach allows for further extraction of other views. In a previous work (Kirschner et al., 2023b), we proposed a method for automatically annotating components of an architectural model with vulnerability information based on the trace link model. The goal of this method is to extract security information from source artifacts and integrate it into an existing architecture-based security model to enable model-based security risk assessment. In addition, our approach allows the composition and refinement of views from artifacts generated at runtime. For example, behavioral views can be integrated from log files, such as measurement logs from monitoring tools. These extracted behavioral views can then be composed and refined using our approach in the same way as behavioral views from static information.

3.5. Composition and refinement of views

The model-driven composition and refinement step links model-providing processes, such as reverse engineering, with model-consuming processes, such as quality prediction. To that end, we have developed and implemented a framework to automate model-driven knowledge transformation for software systems. Our framework is metamodel-independent and can be adapted to different target metamodels. It formulates refinement rules using GPLs such as Java, Xtend, or transformation languages such as QVT or ATL. The framework supports model-consuming processes by linking model-providing processes with model-consuming processes. It consists of three processes: information discovery, information processing (composition and rule-based refinement), and transformation.

```
void call(int x) {
  x += x >> 2;
  x = -x & (x << 16);
  if ((x % 3) == 0) {
    external.foo();
  } else {
    this.bar();
  }
}
```

(a) Source code of a Java method as input to the transformation to map its behavior to a model.



(b) UML-like graphical representation of a view of a method's behavior as a state diagram.

Fig. 3. Sample application of extraction rules to map component behavior to a view of method behavior.

Algorithm 3 Processing pipeline for provided models including discovery, composition, and the rule-based refinement step. The Process Element function applies the refinement rules to the given element and recursively to the resulting implications that need to be processed after the current element has been processed.

function CompositionRefinement(V_e, R)
  $v_q \leftarrow \{\ \}$
  for all $e \in$ Discover(V_e) **do**
    $v_q \leftarrow$ ProcessElement(v_q, e, R)
  end for
  return v_q
end function

function ProcessElement(v_q, e, R)
  $v_q \leftarrow C(v_q, e)$
  for all $r \in R$ **do**
    for all $i \in \mathcal{R}_r(v_q, e)$ **do**
      $v_q \leftarrow$ ProcessElement(v_q, i, R)
    end for
  end for
  return v_q
end function

Algorithm 3 shows the processing pipeline for composing and refining the previously extracted views. In the Discover function, heterogeneous information is retrieved from model-providing processes, classified into named elements and relationships, and translated into the model-consuming domain. The resulting information is provided as a wrapper for the processing phase. The processing phase consists of two main steps: composition, where named elements and relationships are added to an intermediate model and their dependencies are recursively processed, and rule-based refinement, where views are manipulated to conform to specific rules. This comprehensive approach ensures seamless use of model-driven knowledge and improves software evolution processes in modern software systems.

The Discover function extracts the previously heterogeneous views of the system to make them consistent for subsequent processes. This function extracts all elements from the set of views V_e and returns them as a new set of views v_q. The ProcessElement function takes as input the set of all already composed and refined elements, the new element to add to the given set, and the refinement rules to apply to the new element. The function works by first adding the new element to be refined to the given set. Adding the element to the set of refined elements is referred to as composition. In the next step, all given refinement rules are applied to this element. This refinement process can produce new model elements as implications i, which are also processed recursively using the ProcessElement function. The function continues until there are no more refinement rules and resulting implications for the process. At the end, the function returns the set of all refined elements, including the new element.

3.5.1. Composition of views

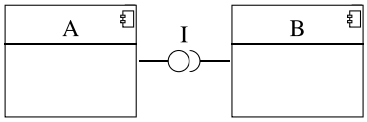
Our model composition combines various model elements into a more comprehensive and complex system view. Let V be a view and e a model element that is not part of that view $e \notin V$. The model elements represented by V and the single model element e are based on the same metamodel and are valid accordingly. The symbol C represents the model composition. If e is not yet a member of the set V, then the model composition is a new view containing all elements of V and e. It is defined as $C(V, e) := V \cup \{e\}$. Identical model elements are composed into a single element. The element name is used to identify the named model element. Two elements with the same name are identical and are composed into a single element, where model composition is a recursive operation that can be applied to all model elements. Relationships to be composed are identified by their type and by comparing the identity of their referenced model elements. As elements are added to the model, they are combined into a richer and more complex system view.

The identity of model elements varies according to their type. Classes are identified by their fully qualified names, which combine the namespace and class name, e. g. `com.example.myproject.MyClass`. Instances are identified by the class and a unique identifier, like an variable name. Interfaces are usually identified by uniform resource identifiers (URIs), e. g. https://example.com/myproject/MyInterface, and other items, such as configuration files, may use the file path as an identifier.

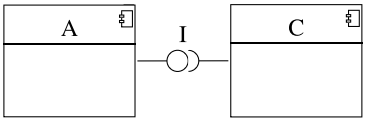
Fig. 4 shows an example of combining a new element into a view. Fig. 4(a) shows the simplest form of relationship between components A and B, where component B requires an interface I, which is provided by component A. Fig. 4(b) shows the view of components A and C before the relationship AB was added. Component C requires an interface I from component A. Fig. 4(c) shows the view created by the composition, which includes all three components, their relationships, AB and AC, and the interface I. In this example, a relationship AB is added to a view of components A and C with its relationship AC, creating a new view with three components. The two components A with the same name in Figs. 4(a) and 4(b) are identical and are combined into one component in Fig. 4(c).

3.5.2. Refinement of views

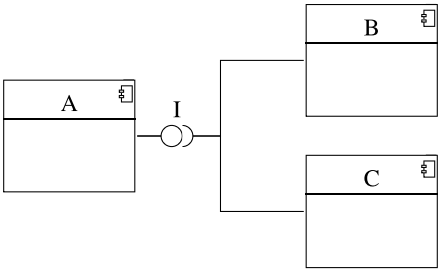
Model refinement describes the automated process of improving the quality of an extracted view by adding more detail or accuracy. The idea is to create models at different levels of abstraction and connect them through refinement to support the process of reverse



(a) The UML assembly connector represents the relationship AB, showing the two associated components.

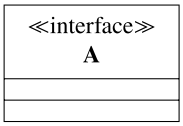


(b) The view shows components A and C with their relationship AC, into which relationship AB will be inserted through composition.

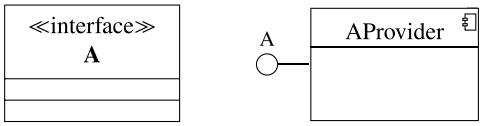


(c) The composed view includes three components, A, B, and C, and their relationships AB and AC.

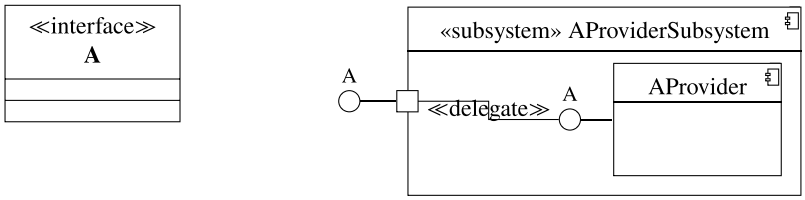
Fig. 4. The UML-like graphical representation illustrates view composition where relation AB is added to a view with components A and C, which already includes relation AC, resulting in a new view containing three components.



(a) In the composition of interface A, no component provides the interface, triggering the refinement process due to unsatisfied rule r_1.



(b) After composing interface A and applying refinement rule r_1, component AProvider is introduced to provide interface A. This composition satisfies r_1 but results in the unsatisfaction of refinement rule r_2.



(c) After composing interface A and applying refinement rules r_1 and r_2, the view satisfies all refinement rules, ending the recursive refinement process initiated by the composition of interface A.

Fig. 5. The UML-like representation illustrates recursive view refinement with refinement rules r_1 and r_2. Rule r_1 requires each interface to be provided by at least one component, while rule r_2 mandates that each component be encapsulated by a subsystem, with interfaces delegated to that subsystem.

engineering. This refinement process can create new model elements called implications. Let V be a view and e be a model element that is part of the view $e \in V$. The model elements represented by V are based on the same metamodel and are valid accordingly. The symbol $\mathcal{R}_r$ represents the model refinement for the refinement rule r. The model refinement is a transformation of the view V into a new view $\mathcal{R}_r(V, e) = V'$. A refinement rule is a model-to-model transformation that checks whether the given model element e in the view satisfies certain rules. If the rule is satisfied, the view remains unchanged. However, if the model element does not satisfy the rule, the transformation creates new model elements called implications. These new implications must also be added to the view in order for it to satisfy the rule.

Fig. 5 shows a UML-like example of a refinement process using two exemplary refinement rules. The first rule is called $r1$ and the second rule is called $r2$. An arbitrary view V satisfies the refinement rule $r1$ if each interface of V is provided by at least one component. The refinement rule $r2$ is satisfied if each component of V is encapsulated by a subsystem. Moreover, the provided interfaces of encapsulated components have to be delegated to the subsystem to satisfy $r2$. Fig. 5(a) shows an arbitrary view V created by the composition of a view with an interface A. After the composition V does not satisfy $r1$ since no component provides A. A refinement for the refinement rule $r1$ is used to transform V to satisfy $r1$. Fig. 5(b) shows the view after the transformation. The new view V' is created by the composition of V and the implications of the refinement for $r1$. The view V' satisfies refinement rule $r1$. A single implication is required to satisfy $r1$. This implication is a component called $AProvider$ providing A. After the first refinement V' does not satisfy $r2$, leading to another refinement for refinement rule $r2$. Fig. 5(c) shows the view V'' created by the composition of V' and the implications of the refinement for $r2$. V'' satisfies $r1$ and $r2$. To satisfy $r2$ an encapsulating subsystem as well as an interface delegation are composed with V' creating V''. This refinement does not break any refinement rule. As a consequence, the recursive refinement process terminates. The result of the refinement process is V'', which is a composition of V with the implications and recursive implications required to satisfy all refinement rules.

3.6. Final transformation of views

Finally, the last transformation process takes the intermediate view composed and refined during information processing and transforms it into one or more output views. These output views can be disjoint and

based on different metamodels. These output views can then be used for system documentation or as input for other model-intensive processes, such as quality prediction. This approach facilitates the integration and unification of architectural descriptions and reduces the necessity for manual intervention. The completion process can be tailored to meet the specific project requirements, including the level of detail and focus on specific architectural patterns.

Algorithm 4 describes a function called Finalization, which takes two arguments, v_q and F. The function is responsible for transforming an intermediate view v_q into one or more output views V_f tailored to the end user's specific needs. The function iterates over all model-to-model transformations $f \in F$ and applies them to v_q, i.e. $\mathcal{F}_f(v_q)$. The output of the transformation $\mathcal{F}_f$ is then added to the set of target views V_f, which is returned at the end. A single transformation takes as input a source view v_q of the system, showing all named elements and relationships. It is based on a transformation definition f that specifies how the elements are transformed into a set of target views V_f. As part of the transformation process $\mathcal{F}_f$, this model-to-model transformation acts like a filter that selects elements $e \in v_q$, from the source view and combines them according to certain criteria into a new target view $\mathcal{F}_f(v_q) \rightarrow v_f \in V_f$.

Algorithm 4 The transformation function is responsible for transforming an intermediate view into one or more output views tailored to the end user's specific needs.

```
function Finalization(v_q, F)
    V_f ← { }
    for all f ∈ F do
        V_f ← V_f ∪ F_f(v_q)
    end for
    return V_f
end function
```

3.7. Retriever illustration

We will use a simplified Java-based project using the Spring Framework[1] and Docker[2] to illustrate the next steps in our reverse engineering process. The Spring Framework is a web application framework. Docker is software for isolating applications through container virtualization. The examples, and especially the source code snippets, have been greatly simplified and adapted for illustrative purposes. Therefore, we will focus on the annotations and leave the other classes aside for now. The example uses a Spring-based `AccountController` class and `AccountServiceClient` interface to illustrate how components are first extracted and later connected using information from two corresponding Docker files. Likewise, the Java source code does not need to be annotated, as our approach allows for a more comprehensive analysis, including examining inheritance structures in Java artifacts.

3.7.1. Identification of artifacts

The process begins by identifying relevant artifacts, which in this case are Java source files, Spring configuration files, and Docker files. These artifacts contain the application's structural and behavioral details. Java files contain the source code that defines the application's behavior and the networking structure of the components. Spring configuration files contain additional details, such as URL mappings and service configurations. Docker files contain the deployment configuration details. The technologies shown are for illustrative purposes only and represent a subset of what is possible. For an overview of additional technologies that are currently supported, see Section 4.

[1] https://spring.io/
[2] https://www.docker.com/

3.7.2. Extraction of components with interfaces and deployments

Fig. 6 shows the sample step for extracting the `AccountController`. The extraction rules recognize it as a REST controller component based on its `@RestController` annotation and its `@GetMapping` method. In this example, the Java class `Account` is a data transfer object that is not directly mapped to a component but is used as a parameter in the provided endpoint. In addition to the Java source code, the extraction process also reads the `account-service.yml` configuration file to determine the context path for the `AccountController`. This context path affects the base URL for all endpoints in the `AccountController`. The extraction rules prepend the context path to the path defined in the `@GetMapping` annotations. This accurately reflects the full URL of each endpoint as it is deployed. These extraction rules were defined for a specific technology stack and can be reused for applications with a comparable technology foundation. In our approach, extraction rules represent a more abstract concept for formalizing additional model-to-model transformations that can be defined for one (project-specific) of the various (technology-specific) systems.

Fig. 7 shows the `AccountServiceClient`, which uses the `@FeignClient` annotation, which is a declarative REST client in the Spring Cloud Netflix library. It is a declarative web service client that simplifies calling RESTful services. The extraction rule identifies the `AccountServiceClient` as a service client component based on the `@FeignClient` annotation and the `@GetMapping` method as a service call.

Fig. 8 shows the Docker environment, in which both the `AccountController` and the `AccountServiceClient` are deployed as separate deployment nodes. Docker configurations are used to define the runtime environment for each component. The first Docker file defines the environment for running the `AccountController`, which provides the REST API. The second Docker file prepares the environment for the `AccountServiceClient`, which acts as a client to the `AccountController`. The extraction rule identifies deployment information for each component based on the contents of the Docker files. As a result, the rule maps an `AccountController` component to one deployment node and an `AccountServiceClient` component to another. This extraction and mapping of these two components is independent of the previous steps for extracting components and interfaces.

3.7.3. Refinement and composition

Fig. 9 shows one composition rule and two refinement rules, as well as the resulting architectural view after the rules have been applied.

Once extracted, the previously independent views can be merged. The previously independent views of Docker deployment information are merged with the application architecture extracted from the Java files. Specifically, this step associates the Docker containers with Java classes or Spring components. This is accomplished by merging all components with the same fully qualified name in the composition rule. Once the extracted views are merged, further refinement rules can be applied. To refine the architectural model, we need to ensure that the components that need to communicate are connected in a way that allows this interaction. First, the provided and required interfaces are connected because they point to the same URL. This connection leads to further refinement so that, in the second step, the associated deployment nodes are also connected. First, the `AccountServiceClient` is connected to the `AccountController` through its interface with the refinement rule. This is based on matching paths and host names. Since there is now a direct connection between two components on different deployments, a second refinement rule must be applied to connect these deployment nodes as well.

```
@RestController
public class AccountController {
  @GetMapping(path = "/{name}")
  public Account getAccount(String name) {}
}
```

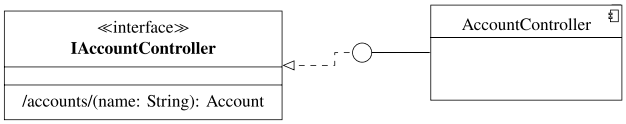
(a) AccountController.java

```
if (cl.isAnnotated("RestController")) {
  Component co = createComponent(cl.name);
  for (Method m :
       cl.annotatedWith("GetMapping")) {
    String path = config.contextPath +
         m.getValue("GetMapping");
    co.addEndpoint(fullPath, m.name);
  }
}
```

(b) Example of an extraction rule for a REST controller.

```
server:
 servlet:
  context-path: /accounts
```

(c) account-service.yml



(d) View of a component and its interface.

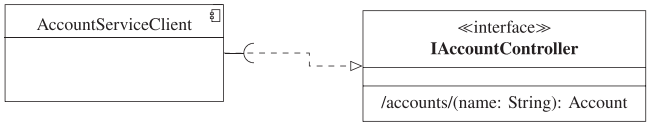
Fig. 6. The `AccountController`, annotated as a REST controller with `@RestController` and `@GetMapping`, is identified as a component. The context path from the `account-service.yml` is prepended to the endpoint paths, indicating the base URL for all related requests.

```
@FeignClient(name = "account-service")
public class AccountServiceClient {
    @GetMapping(value = "/accounts/{name}")
    String getAccount(String name);
}
```

(a) AccountServiceClient.java

```
if (cl.isAnnotatedWith("FeignClient")) {
    Component co =
         createComponent(cl.name);
    for (Method m :
         cl.annotatedWith("GetMapping")) {
        co.addServiceCall(m.getValue("GetMapping"),
             m.name);
    }
}
```

(b) Example of an extraction rule for a REST client.



(c) View of a component and its interface.

Fig. 7. The `AccountServiceClient` is identified as a service client component, using the `@FeignClient` annotation from Spring Cloud Netflix, simplifying RESTful service calls by abstracting HTTP request and response details.

```
FROM java:8-jre
ADD ./target/account-service.jar /app/
CMD ["java", "-jar",
     "/app/account-service.jar"]
```

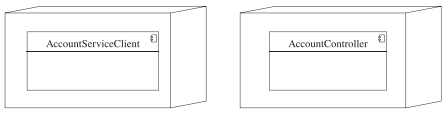
(a) Example Dockerfile for AccountController

```
FROM java:8-jre
ADD ./target/account-service-client.jar
     /app/
CMD ["java", "-jar",
     "account-service-client.jar"]
```

(b) Example Dockerfile for AccountServiceClient

```
for (Dockerfile d : allDockerfiles()) {
    Deployment node =
         createDeployment(d.parentFolder,
         d.name);
    node.addComponents(d.app.name);
}
```

(c) Example of an extraction rule for Dockerfiles.



(d) View of two independent deployment nodes, each with one component.

Fig. 8. In the Dockerized setup, the `AccountController` and `AccountServiceClient` are deployed on separate nodes, with Docker files defining each component's environment. The extraction rule assigns each component to a deployment node based on Docker configurations.

3.7.4. Final architectural representation

In Fig. 10, the refined and assembled model is converted into an output view tailored to the user's specific needs, such as a UML diagram. The resulting architecture model now visualizes `AccountController` and `AccountServiceClient` as connected components, with the interface showing the communication path derived from the Java code and Spring configuration files, as well as the refined details of the Docker deployment. For this goal, a finalization rule is applied to create a UML diagram that maps the components as UML components. We use UML diagrams only to illustrate our approach. However, our finalization rules are UML-independent, and we use the Ecore-based PCM for software architecture models in addition to PlantUML.

```
for (Component c : allComponents()) {
    for (Deployment d : allDeployments()) {
        if (d.contains(c.name)) {
            link(d, c);
        }
    }
}
```

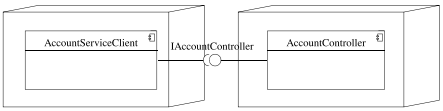
(a) Example Composition Rule

```
for (Component cl : allComponents()) {
    if (cl.hasRequiredInterface()) {
        Interface r = cl.requiredInterface;
        for (Component co :
             allComponents()) {
            if
                 (co.hasProvidedInterface(r))
                 {
                linkInterfaces(co.providedInterface,
                     r);
            }
        }
    }
}
```

(b) Refinement for links between components.

```
for (Deployment d : allDeployments()) {
    for (Component c : d.allComponents()) {
        if (c.hasLinkToOtherComponent()) {
            link(d,
                 c.getLinkedDeployment());
        }
    }
}
```

(c) Refinement for links between deployments.



(d) View of two independent deployment nodes, each with one component.

Fig. 9. After component extraction, Docker deployment information is merged with architectural data from Java files and then refined by connecting components.

```
for (Component component : allComponents()) {
    UMLComponent umlComponent = createUMLComponent(component.getName());
    for (Endpoint endpoint : component.allEndpoints()) {
        UMLEndpoint umlEndpoint = createUMLEndpoint(endpoint.getPath());
        umlComponent.addUMLEndpoint(umlEndpoint);
    }
    Deployment deployment = component.getDeployment();
    UMLDeployment umlDeployment = createUMLDeployment(deployment.getName());
}
```

Fig. 10. Example UML finalization rule.

4. Technical realization

To promote transparency and accessibility, we have released our implementation of the Retriever approach as open-source on GitHub (Kirschner et al., 2023) under the Eclipse Public License 2.0 (EPL2). This allows researchers, developers, and practitioners to access the code, contribute to its development, and adapt it to their specific needs. The detailed implementation, which includes all the details of the extraction, composition, and refinement rules used for the case studies, is also available in the open-source repository. Our approach is implemented using the Eclipse Modeling Framework and is available as a plug-in to the Eclipse platform. The technical implementation of our approach combines third-party libraries for input processing, leverages the PCM metamodel for detailed system analysis, and provides flexibility by outputting both PCM and UML software architecture models. Our approach is available as an open-source Eclipse product with both graphical and command-line interfaces. We have also implemented the command-line interface as GitHub Actions for use in continuous integration (CI) and continuous delivery (CD) pipelines. With these GitHub Actions, our approach can be easily integrated into CI/CD pipelines on GitHub and, for example, automatically generate both PCM and PlantUML models for the existing artifacts as output on every commit. The integration enables the automatic generation of quality prediction models directly from the repositories for each code commit. It also demonstrates a practical application of continuous architecture review and documentation that is consistently integrated into CI/CD pipelines.

Fig. 11 shows the approach schematically and describes which parts process which information. The arrows represent the data flow, and the input and output data are represented as UML notes whose labels

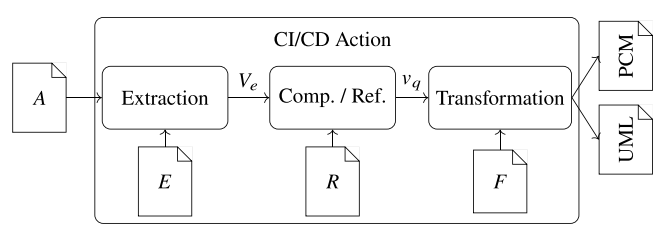


Fig. 11. Schematic of the approach with three main steps, each processing different information.

correspond to those in Fig. 2. The rounded rectangles represent the three main steps of our approach. These three steps are described in more detail in the following three subsections. The large rounded rectangle represents deployment as a GitHub Action, to make it easy to use our approach in the build, test, and deploy pipeline on GitHub. This diagram is based on the architecture of our Retriever implementation. More detailed information, including architecture diagrams of the individual components, can be found in Kirschner et al. (2023).

4.1. Artifact extraction implementation

Our approach relies on various libraries to facilitate the initial discovery of artifacts. For example, our implementation currently includes a Java source code finder based on the Eclipse Java Development Tool and supporting Java 21 archives and properties files, an ECMAScript 6 finder based on the OpenJDK Nashorn engine, and a SQL statement

finder based on the JSqlParser. There are also finders for JSON, XML, and YAML configuration files, also based on various third-party libraries. The strategic use of established artifact extraction libraries is preferred because of their efficiency and reliability. Not only are these libraries regularly updated and maintained to ensure compatibility with ongoing technology developments, but they also provide prebuilt functionality that saves development time and is often improved through community efforts and rigorous testing. This approach not only streamlines the development process but also ensures consistency across project components.

The extraction rules for model-to-model transformations take the models generated by the finders as input. For example, a Java finder identifies and analyzes Java source code artifacts, while the Spring extraction rules transform the models generated by the Java finder into initial views based on the technology knowledge of the Spring framework. The extraction rules are implemented using Xtend. We use Xtend, a general-purpose programming language that follows imperative and functional programming paradigms and allows flexible rule structuring with both imperative and declarative transformation definitions. Xtend, a versatile and eloquent dialect of Java, combines the strengths of both Java and script languages. We implemented extraction rules for the following technologies: Docker, EcmaScript, Gradle, JaxRS, Maven, and Spring. We preselected these technologies because they are among the most widely used technologies for component-based RESTful Web services Stack Overflow (2023), fortiss (2023) and contain important architectural infottrmation. However, our approach also allows adding additional extraction rules for already supported or additional technologies that can be dynamically loaded at runtime.

4.2. Composition and refinement implementation

Our approach uses the PCM metamodel for processing. The PCM metamodel provides a standardized representation of software architectures, making it a suitable choice for our purpose. It allows us to capture essential architectural concepts such as components, connectors, and interfaces. The PCM serves as an intermediate representation for combining the extracted information from the extraction transformations for the composition and refinement of views. However, the concepts of our approach are also applicable to other ADLs. The model-to-model transformation rules are implemented in Java. We use Java, a general-purpose programming language that follows imperative and object-oriented programming paradigms and allows easy structuring of imperative rules using object-oriented concepts such as inheritance for transformation definitions. The choice of Java was influenced by its compatibility with our composition and refinement framework, which is also Java-based. This compatibility allows for better integration and uniformity of our components. It also simplifies maintenance tasks because the entire code base is in a single language. The underlying framework also allows the integration of transformation languages such as Xtend, ATL, or QVT. We have implemented these refinement rules generally for the domain of component-based systems, i. e. they can be reused in any system within this domain. They can, therefore, be combined with any extraction rules and are technology-independent. For the refinement implementation, the following seven refinement rules have been implemented in the framework:

r_1 := One or more interfaces specify a signature.
r_2 := One or more components provide an interface.
r_3 := A component has a service effect specification for each signature it provides.
r_4 := A component is associated with at least one container.
r_5 := An assembly of components requires all associated containers to be linked.
r_6 := A container link is specified by exactly one link specification.
r_7 := A link specification is associated with one or more container links.

4.3. Final views implementation

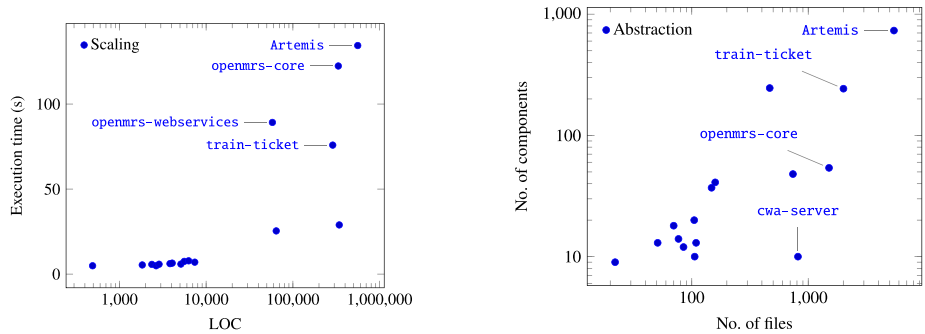
For the final transformation of views, our approach uses both PCM and PlantUML models as output. The PCM output captures information specific to software quality and architectural aspects, allowing users to perform model-based quality analysis. The PlantUML output, on the other hand, provides a broader view of the software system's architecture and is used to create UML diagrams that provide a more widely recognized graphical representation of the system's structure. We have implemented the final view transformation based on finalization rules for outputting views based on the PCM and UML metamodels. The concepts of these finalization rules are also applicable to other ADLs. The implementation of additional finalization rules would be necessary to support using other UML tools. It is of utmost importance to adapt these rules to the specific metamodel of the UML tool. This would require mapping the concepts of the ADL to those of the UML tool and defining the rules for transforming the ADL views into UML diagrams. These finalization rules are implemented in the domain of component-based systems, i. e. they can be reused in any system within this domain. They can, therefore, be combined with any extraction and refinement rules and are technology-independent.

5. Evaluation

We structured our evaluation based on the goal question metric (GQM) approach (Basili et al., 1994). The GQM approach is a goal-oriented method for improving and measuring software quality. It defines a three-stage measurement model: 1. A goal is set for an object considering various reasons, quality models, perspectives, and specific environments. 2. A set of questions is used to define models of the object under study and to characterize the assessment or achievement of a specific goal. 3. Each question is associated with a set of metrics to answer it measurably.

While the evaluation question aims to assess the value of the effectiveness of the *Retriever* approach, the research question aims to generate new knowledge or understanding about a topic. This approach ensures that measurements are top-down, goal-oriented, and model-based, providing a framework for interpreting data against stated goals. The following GQM plan outlines our evaluation goals, questions, and metrics:

G1: Applicability of the *Retriever* approach to relevant software systems to generate an abstraction of the system as an architectural model. *Q1.1:* Is the *Retriever* approach applicable to relevant software systems with different technologies? *(M1.1.1)* The relevance of GitHub projects is measured by four key metrics: commits, contributors, forks, and stars. These metrics serve as valuable indicators of a project's popularity and distribution. They can help evaluate a project's relevance and determine whether it is worth using or contributing to. More commits indicate active maintenance and a large developer base. More contributors indicate a robust developer community and a higher likelihood of ongoing maintenance. More forks indicate that other developers find the project useful or interesting. More stars indicate widespread user interest. *(M1.1.2)* For technology usage, two metrics are used: the number of lines of code (LOC) (excluding whitespace and comments) and the number of source files. A larger code base may indicate increased complexity, making maintenance and readability more difficult. More source files could indicate a highly modular system, increasing the difficulty of MDRE. *(M1.1.3)* For system views, the chosen metric is the number of different technologies, each providing a unique perspective on the system. More technologies can increase view complexity, making MDRE more difficult. *Q1.2:* To what extent is the approach scalable and transferable to larger real systems in the context of industrial applications? *(M1.2.1)* In the context of



(a) The ratio of the average execution time in seconds to the sum of the LOC for all artifacts analyzed in the case study. The four case studies with the highest execution times are labeled.

(b) The ratio of the number of source files to the number of components in the architectural model, indicating the level of abstraction. The four case studies with the highest number of source files are labeled.

Fig. 12. Evaluation results for the applicability and scalability of the approach to generate an abstraction of the larger real systems as a model.

industrial applications, the first metric is the cumulative number of LOC across all artifacts of the software system considered in our approach. The complexity of this code can potentially hinder the speed of reverse engineering processes. *(M1.2.2)* In terms of scalability, the second metric is the average execution time of GitHub Actions for comprehensive model extraction. A fast reverse engineering tool can speed up results, saving engineers time and resources. *Q1.3:* Does the *Retriever* approach lead to valid architectural models that abstract from the source code? *(M1.3.1)* When assessing the validity of architectural models, the primary metric is the number of violated Object Constraint Language (OCL) constraints in the reverse-engineered models. OCL constraints, which are conditions built into EMF models to delineate constraints and limitations, signal potential architectural defects when violated. *(M1.3.2)* Regarding abstraction, the secondary metric is the ratio of source files to the number of components in the architectural model. This ratio serves as an abstraction indicator and is critical in evaluating the reduction quality of architectural models.

G2: Accuracy of the *Retriever* approach in generating both structural and behavioral properties of relevant software systems, forming a software architecture model together. *Q2.1:* Have the structural properties of the component-based software architecture been correctly identified? *(M2.1.1)* The metrics used to measure this are precision, *(M2.1.2)* recall, and *(M2.1.3)* F_1 score for the components, connectors, and provided and required interfaces. *Q2.2:* Are the component-based software architecture behavioral properties (SEFF) correctly identified so that data and control flows exist across component boundaries that are not connected at compile time? The external calls are the link between the structure and the behavior view. Therefore, they indicate whether the approach could correctly extract and combine these two view types. *(M2.2.1)* The metrics used to measure this are precision, *(M2.2.2)* recall, and *(M2.2.3)* F_1 score for the external calls in the SEFF.

5.1. Applicability to relevant software systems

For the first goal of the evaluation, we reviewed statistics (Stack Overflow, 2023; fortiss, 2023) that provide information about the most popular and widely used technologies for implementing and deploying web services systems. Based on these statistics, extraction rules were implemented for the following technologies: 1. Maven and Gradle for build automation. 2. Docker for container virtualization. 3. Spring MVC and Boot for web application development. 4. EcmaScript for web client application implementation. 5. Jakarta RESTful web Services (JaxRS) for RESTful web services implementation.

We chose these technologies not only because they are among the most widely used domain technologies for modern component-based systems, such as web services or microservices. We also chose these technologies because they provide established frameworks that may contain important architectural information that can be recovered using our extraction rules. In addition, these technologies have different concerns and each contributes a different view of the system. The rules for Spring MVC and Boot have also been implemented in a common extraction rule because the rules for both technologies are very similar. The Spring and JaxRS rules are both Java-based technologies, so they use the same artifacts as input.

The rule implementations described in Section 4 are used for this evaluation. The extraction rules in the case studies were designed to identify and map significant architectural elements from various software artifacts, such as source code and configuration files. These rules were technology-specific, considering the unique characteristics and semantics of each technology stack used in the case studies. Composition rules were used to integrate the extracted architectural elements into a coherent model. Refinement rules improved the accuracy and completeness of the architectural models by adding or adjusting elements to reflect more profound insights or correct initial extractions. Finalization rules were implemented to output views based on the PCM and PlantUML metamodels.

Based on these relevant domain technologies, relevant systems for the evaluation scenarios were identified. This selection was based partly on related work and partly on open availability. Based on open availability, the evaluation scenarios were further selected based on their popularity on GitHub. The systems selected for evaluation are listed in with their GitHub identifiers, distribution metrics, technology metrics, and metrics for scalability and abstraction of our retriever approach.

For these open-source case studies, our approach was applied as a GitHub Action within a CI/CD pipeline, as described in Section 4. The automatically generated architectural models were first checked for syntactic correctness. For the textual notation of the UML diagrams, PlantUML was able to generate the appropriate diagram for each case study without error messages. As the PCM models are much more extensive, they were validated using the EMF validation framework. In addition, validation was performed using OCL constraints to ensure the syntactic correctness of the models. The PCM instances also completed model validation in Eclipse with no errors or warnings.

To evaluate the scalability of our approach on larger systems, we performed a local test on a laptop with an i7-8550U processor, 16 GB of RAM, and Windows 11 as the operating system. For each case study, we independently executed the action 10 times with the same inputs and recorded the execution times for each run. The differences in execution times were negligible, always less than one second. To verify

Table 1
Evaluation results regarding the applicability of the approach to relevant software systems to generate an abstraction of the system as an architectural model.

System-ID	Distribution		Technology (files/code)			Reverse engineering	
acmeair	Commits	78	Ecma	5	708	Execution time	7.028 s ± 0.044 s
	Contributors	2	Gradle	2	75	Components	20
	Forks	154	Java	81	6 207		
	Stars	131	Maven	1	56		
			Xml	16	349		
microservices-basics-spring-boot	Commits	48	Docker	9	126	Execution time	6.166 s ± 0.141 s
	Contributors	5	Ecma	8	153	Components	10
	Forks	437	Gradle	9	691		
	Stars	692	Java	37	1 385		
			Xml	9	182		
			Yaml	34	1 294		
java-microservice	Commits	143	Docker	4	97	Execution time	7.496 s ± 0.165 s
	Contributors	1	Java	116	3 147	Components	41
	Forks	305	Maven	11	1 376		
	Stars	338	Xml	9	233		
			Yaml	19	743		
blog-microservices	Commits	239	Docker	7	39	Execution time	5.825 s ± 0.122 s
	Contributors	1	Gradle	12	663	Components	14
	Forks	308	Java	37	1 401		
	Stars	400	Xml	2	31		
			Yaml	19	731		
cwa-server	Commits	1 664	Docker	5	170	Execution time	28.925 s ± 0.691 s
	Contributors	30	Java	615	40 602	Components	10
	Forks	386	Json	48	24 554		
	Stars	1 894	Xml	31	273 343		
			Yaml	117	3 638		
cwa-verification-server	Commits	211	Docker	1	8	Execution time	5.920 s ± 0.115 s
	Contributors	30	Ecma	1	40	Components	12
	Forks	107	Java	58	3 618		
	Stars	349	Json	1	409		
			Maven	1	142		
			Xml	1	294		
			Yaml	22	610		
microservice	Commits	143	Docker	13	43	Execution time	5.724 s ± 0.072 s
	Contributors	8	Java	42	1 833	Components	18
	Forks	348	Maven	7	380		
	Stars	692	Yaml	8	110		
train-ticket	Commits	1 186	Docker	46	249	Execution time	75.867 s ± 0.858 s
	Contributors	18	Ecma	992	197 869	Components	243
	Forks	191	Gradle	2	13		
	Stars	605	Java	635	37 738		
			Json	170	33 274		
			Maven	45	2 229		
			Xml	1	30		
			Yaml	114	16 619		
apache-spring-boot-microservice-example	Commits	7	Ecma	1	36	Execution time	4.921 s ± 0.093 s
	Contributors	1	Java	15	326	Components	9
	Forks	10	Maven	3	108		
	Stars	9	Yaml	3	21		
Artemis	Commits	17 527	Docker	5	113	Execution time	134.416 s ± 0.985 s
	Contributors	30	Ecma	2680	239 802	Components	733
	Forks	262	Gradle	12	776		
	Stars	383	Java	1984	216 149		
			Json	480	88 131		
			Maven	11	896		
			Xml	140	8 103		
			Yaml	126	5 173		
spring-cloud-movie-recommendation	Commits	6	Java	37	931	Execution time	5.396 s ± 0.140 s
	Contributors	1	Json	3	309	Components	13
	Forks	11	Maven	6	467		
	Stars	15	Xml	5	128		
openmrs-core	Commits	18 874	Docker	1	79	Execution time	122.385 s ± 0.737 s
	Contributors	30	Ecma	21	4 346	Components	54
	Forks	3 504	Java	1874	189 296		
	Stars	1 273	Maven	21	4 566		
			Xml	308	134 587		
			Yaml	10	612		

(*continued on next page*)

Table 1 (*continued*).

System-ID	Distribution		Technology (files/code)			Reverse engineering	
openmrs-module-webservices.rest	Commits	1 730	Ecma	3	79	Execution time	89.202 s ± 0.662 s
	Contributors	30	Java	672	53 997	Components	48
	Forks	492	Json	6	503		
	Stars	76	Maven	14	2 253		
			Xml	43	1 182		
			Yaml	1	52		
spring-petclinic-microservices	Commits	718	Docker	3	26	Execution time	6.466 s ± 0.407 s
	Contributors	30	Ecma	21	263	Components	13
	Forks	1 983	Java	46	1 216		
	Stars	1 483	Json	12	816		
			Maven	8	861		
			Xml	3	579		
			Yaml	16	300		
piggymetrics	Commits	290	Docker	10	58	Execution time	7.824 s ± 0.165 s
	Contributors	13	Ecma	8	1 664	Components	37
	Forks	5 987	Java	97	3 292		
	Stars	12 740	Maven	10	684		
			Yaml	23	588		
WebGoat	Commits	2 998	Docker	1	29	Execution time	25.411 s ± 0.674 s
	Contributors	30	Ecma	90	44 099	Components	246
	Forks	4 731	Java	355	18 201		
	Stars	6 120	Json	5	181		
			Maven	1	852		
			Xml	5	582		
			Yaml	10	317		

the plausibility of our results, we compared the execution times of our local test with those of the GitHub action run as a cloud-based solution on a Linux virtual machine. The scalability of the GitHub action execution times was comparable to the local execution times. Fig. 12(a) compares the average execution time for each case study with the size of the input data, which is the sum of the LOC for all relevant artifacts analyzed in . The execution times are plotted on the linear y-axis in seconds and the number of LOC is plotted on the logarithmic x-axis. As expected, the average execution time increases with the size of the input data. Thus, the case study with the longest execution time is the one that requires the most LOC to analyze.

To evaluate the abstraction capability of our approach for larger systems, we compare the number of source files to the number of components in the final architectural model. This ratio serves as an indicator of the reduction quality of the reverse-engineered architectural models. The number of source files was taken as the sum from . Fig. 12(b) shows the ratio of the number of source files to the number of components. The number of source files is plotted on the logarithmic y-axis and the number of components is plotted on the logarithmic x-axis. As expected, the number of components increases with the number of source files. For example, our largest case study with 5438 source files was reduced by 86.52% to 733 components.

5.2. Accuracy

For the second goal of the evaluation, we first manually created a software architecture model. This architecture model serves as a reference (gold standard) for comparing the automatically generated architecture models with our retriever approach. It should provide information about the expected components, interactions, interfaces, dependencies, and connections in the case studies. To create an accurate software architecture model for a component-based open-source software system, we performed the following 5 iterative steps for each case study. These steps combine system understanding, architectural style selection, visual representation, pattern considerations, verification, documentation, and iterative refinement to produce a comprehensive and reliable architectural model.

1. Understand the system and identify its components: Obtain a comprehensive understanding of the system's functionality, purpose, users, and tasks through documentation, source code, design documents, and other publications. Identify key components through code analysis, documentation, and available resources.
2. Select an appropriate architectural style and define the component model: Select an architectural style (e. g., microservices, layered architecture) that meets the system requirements. Define rules and conventions for developing, deploying, and operating components. Specify interface syntax, semantic, lifecycle, deployment, and runtime environment.
3. Create a component diagram: Represent components, interactions, interfaces, dependencies, and connections in a component diagram. Use standard notation (e. g., UML) or custom notation to ensure clarity and consistency. Use tools such as PlantUML to visualize the system architecture.
4. Consider architectural patterns: Examine common software architecture patterns (e. g., n-tier, client-server, microservices) and how they can improve system properties. Integrate these selected patterns into the component model and architectural style.
5. Verification and Refinement: Obtain stakeholder feedback to verify the architectural model and update it based on their input to ensure alignment with evolving system requirements and constraints, thereby mitigating potential biases.

    (a) Stakeholder Engagement: Collaborated with software developers involved in other case studies to review the architectural model. This facilitated broad discussion and gathered critical feedback that helped refine the architecture, reduce bias, and ensure alignment with industry practices.
    (b) Documentation Engagement: Thoroughly analyzed the (architectural) documentation provided by the original case study developers. The goal of comparing the architectural model to the original architectural diagrams was to resolve inconsistencies and align the model more closely with the documented intent, effectively reducing inherent bias.

Each of the five steps is applied to a subset of case studies selected based on the availability of architectural documentation. This documentation is essential for the manual creation of architectural models and allows the identification of components, interactions, interfaces, dependencies, and connections. Comparing our UML models to existing diagrams in the case studies helps reduce bias. The seven selected case

Table 2
Comparison results and metrics for the accuracy of the *Retriever* approach in generating both structural and behavioral properties.

System-ID	Structural			Behavioral		
	p	r	F_1	p	r	F_1
acmeair	0.81	0.87	0.84	0.75	0.57	0.65
microservices-basics-spring-boot	0.90	0.64	0.75	0.83	0.63	0.71
microservice	0.78	0.88	0.82	0.82	0.82	0.82
apache-spring-boot-microservice-example	**1.00**	0.89	**0.94**	0.63	0.71	0.67
spring-cloud-movie-recommendation	0.80	0.73	0.76	0.83	0.59	0.69
spring-petclinic-microservices	**1.00**	0.76	0.87	**0.94**	0.83	**0.88**
piggymetrics	0.89	**0.91**	0.90	0.69	**0.85**	0.76

studies are detailed in Table 2. In addition, we refined the gold standard architecture model through discussions with software developers involved in other case studies.

We then compare the manual model with the model from our automated reverse engineering approach and categorize the comparison result to answer the questions related to the second goal of the evaluation plan. This subset allows for comparison with manually constructed architectural models, evaluates the accuracy of the automated approach, and reflects the variability and diversity of the case studies. The goal is to assess the accuracy of the reverse engineering approach and to identify potential inaccuracies and shortcomings in the reverse engineering results. The method of comparison involves contrasting the structural and behavioral properties of the manually analyzed component architecture with the corresponding reverse-engineered component architecture. The following elements are compared in the models: The boundaries of components and their groupings within components and nodes. Interfaces provided and required by components, along with their corresponding URIs. Relationships of calls between components via external calls, as specified in the SEFF.

The metrics used for measurement include precision p, recall r, and F_1 score. True positives are instances where the model elements identified in the manual analysis are also detected in the reverse engineering results. False positives are instances where model elements are discovered in reverse engineering but not in the manual analysis. False negatives are instances where model elements are found in the manual analysis but not in the reverse engineering. After categorizing the comparison results, the metrics are calculated for each case. These metrics can be used to evaluate the accuracy of the *Retriever* approach. Table 2 shows the detailed comparison results and metrics for each case.

5.3. Results and discussion

Because reverse engineering large code bases results in large models, scalability becomes a critical factor. Our approach has already demonstrated its effectiveness in real-world projects ranging in size from small to large. As expected, the average execution time increases with the size of the input code base. However, only a linear increase is observed for the case studies. Furthermore, the execution time for the largest case study with over 559143 LOC is still very reasonable at 134.416 s. However, additional efforts are needed to improve the performance for larger systems. To this end, we intend to explore various techniques, such as parallelizing model transformations. In future work to reduce execution time, transformation rules will be executed concurrently whenever possible. The ratio of source files to the number of components in the final architectural model serves as an indicator of the our approach. It demonstrates that our approach significantly reduces the number of source files to a manageable number of components, contingent on technology-specific extraction rules. The graph indicates that the approach abstracts from the existing files and does not represent them 1:1, indicating that the approach produces valid architectural models that abstract from the source code.

The second goal was to evaluate the accuracy of the approach in generating both structural and behavioral views. The results in Table 2 indicate that the approach is capable of recognizing most of the primary components of a software system and their interrelationships. For example, F_1 values for structural properties range from 0.75 to 0.94. In addition, the method can effectively analyze the internal structure of a component by identifying its subcomponents and their relationships. This is illustrated by the F_1 values of the behavioral attributes, which range from 0.65 to 0.88. However, the approach falls short in identifying components that do not directly contribute to the functions of the system domain, such as configuration-related components. At the same time, our approach often results in inadequate analysis of component interfaces, leading to incorrect or redundant paths and incorrect mappings between interfaces and components. The observed variation in precision values from 0.75 to 0.94 across the case studies can be attributed to several factors, including the variability of the technologies, the specificity of the extraction rules, and their alignment with the technologies used. This indicates that technologies with more rigorous and structured implementation requirements tend to yield higher precision due to more accurate analyses. Similarly, discrepancies in recall are primarily due to the high incidence of false negatives, especially in larger and more complex systems. This suggests that these systems are vulnerable to the omission of their subtle or less overt architectural elements by generic extraction rules, resulting in an incomplete identification of all relevant elements. These results suggest that the accuracy of the approach is primarily dependent on the initial phase of the extraction transformation, which converts artifact models into architectural views. Since the evaluation used only generic technology-specific rules for all extraction transformations, these rules significantly influence the result. The results indicate that frameworks such as Spring provide a better basis for technology-specific extraction rules than frameworks that enforce less stringent implementation requirements, such as JaxRS. A suboptimal result from the *Retriever* approach could signal a poorly structured system that lacks patterns. To remedy this, architects can formulate new project-specific rules. For instance, regular expressions can be readily implemented to identify recurring naming patterns within the system. Moreover, more specific rules could be employed to detect dependencies based on the utilization of global variables, application programming interface calls, or even comments that indicate connections between code segments. In previous work, we have shown that the result of the extraction can be improved by project-specific rules (Kirschner et al., 2023a).

5.4. Threats to validity

We understand the concerns about the generation of extraction rules and the implications for the practical adoption of our approach. We acknowledge that the extraction rules used in our approach are manually defined, which may be a drawback. However, this approach allows us to fine-tune the rules to the specifics of the technologies and artifacts involved in software systems, which is a significant advantage. We understand that manually defining these rules may be perceived as labor-intensive. However, we have found that these rules can be reused across applications with similar technological underpinnings, potentially offsetting this disadvantage once defined. In our current work, we have observed that many applications within the same domain or technology stack share enough similarities to allow for some

rule reuse. This reusability can potentially reduce the initial setup effort of our proposed approach when applied to similar contexts. However, we recognize the importance of addressing concerns about the effort and complexity of defining these rules for different applications.

To this end, we have outlined plans for future research that include an empirical study to evaluate the effort and complexity involved in defining these rules. For this, we have structured our threats to validity based on the guidelines for case study research from Runeson and Höst (Runeson and Höst, 2008).

Internal Validity: The evaluation aims to accurately capture the architecture of each reference system by analyzing source code, configuration files, and expert knowledge. The evaluation compares the constructed architecture with existing documentation and architecture diagrams to ensure that only the expected components and interfaces with their data types are identified. However, the approach to identifying components and their interfaces provides only one possible view of the system. Subjective differences in the manual creation of gold standards can lead to inconsistent results. To address this, guidelines have been developed to standardize the interpretation process to increase consistency and reduce subjectivity. In addition, gold standards are compared to existing architectural documentation provided by the original developers to ensure alignment and accuracy. Platforms such as Eclipse can behave differently under different conditions or configurations, potentially affecting the reliability of the generated models. To mitigate this risk and ensure consistency, a well-defined and consistent configuration is used for the Eclipse platform. This approach facilitates the comparability of models across different analyses.

External Validity: The generalizability of findings from the case studies to systems not included in the study, especially those using different technologies or architectural styles, may be limited. The *Retriever* approach is designed to interpret software artifacts according to their architectural role. Nevertheless, the evaluation was constrained to object-oriented and structured programming paradigms. Future research should investigate the applicability of the approach to other paradigms. The evaluation focused on the specific reference systems and the performance of the *Retriever* approach in the context of web services. To avoid overfitting, only external case studies were used. However, the number of case studies is limited, and we restricted ourselves to a handful of technologies. Nevertheless, they are heterogeneous systems that are representative of the class of systems we are interested in. The case studies were chosen to cover a wide range of different technologies and to be as diverse as possible. In the future, we plan to apply the approach to more external case studies and to add extraction rules for more technologies.

Construct Validity: Discrepancies between the metrics used and the actual goals of the architectural search process could lead to misinterpretations of the effectiveness of the approach. To mitigate this risk, a rigorously defined evaluation approach, specifically the GQM approach, is used. This approach ensures that the evaluation results are closely related to the effectiveness of the approach in achieving the research goals. In our approach, this is the relationship between metrics and goals. Using a specific evaluation approach such as GQM reduces the risk because it shows the relationship between goals and metrics. The evaluation goals and metrics should ensure that the analyzed elements contribute to the achievement of the research goals. The evaluation process involves comparing the expected architecture with the extracted architecture, measuring precision, recall, and F_1 score as metrics commonly used in related reverse engineering approaches.

Reliability: Variations in environmental conditions, such as tool versions, can lead to discrepancies in the replicability of study results. To address this, a detailed description of the software environment, tool versions, and configurations used is provided. This allows other researchers to replicate the study under identical conditions. In addition, all datasets, tools, and source code are publicly available to ensure the reproducibility of the study. The text describes the steps taken to identify and extract the architecture, and presents the evaluation results in terms of LOC, execution time, precision, recall, and F_1 score. Uniform metrics are used for the result, avoiding subjective interpretations and increasing reproducibility. This information allows other researchers to understand and potentially replicate the evaluation process and results. However, evaluation results related to the reference architectures of the case studies used may depend on the experience of the researcher and may be subjective. For this reason, we are publishing a data set (Kirschner et al., 2024) that includes all of our reference architectures, software architecture models, and raw data, and the source code will be publicly available.

6. Related work

The work most closely related to our approach relates to rule-based and model-driven reverse engineering as well as view-based development and is discussed in more detail in the following three sections. We differentiate three categories: approaches solving a similar problem with a different technique, applying a similar technique to a different problem, or addressing a similar problem with a similar technique.

6.1. Solves a similar problem using a different technique

Garcia et al. note that the dominant method for automated architecture reconstruction is the clustering of software entities (Garcia et al., 2013). They present a novel machine learning-based technique for recovering an architectural view that includes a system's components and connectors (Garcia et al., 2011). However, this approach does not allow for the integration of existing knowledge about specific technologies and the resulting views, or the combination of technology views into a single architectural view. Tzerpos and Holt present ACCD (Tzerpos and Holt, 2000), a communication-driven clustering algorithm that organizes files based on naming patterns. Andritsos et al. present LIMBO (Andritsos et al., 2004), a software clustering algorithm based on information theory that uses hierarchical clustering based on the similarities of groups of files to reverse engineer an architecture. Mitchell et al. present Bunch (Mitchell and Mancoridis, 2006), a tool that generates decompositions using similarity measures and uses upscaling algorithms to cluster files based on coupling and cohesion. Jordan et al. present AutoArx (Jordan et al., 2022), an approach that automates the retrieval, integration, and co-evolution of architectural data, facilitating the creation and maintenance of a digital twin that is persistently and autonomously updated as the system evolves. Despite their different principles, all of these reverse engineering methods divide source code units into mutually exclusive clusters based on a dominant principle such as cohesion and coupling or naming patterns, using homogeneous source code as input. Garzón et al. propose a method for reverse engineering object-oriented code into a unified language for both object-oriented programming and modeling (Garzón et al., 2014). Cai et al. propose a new perspective on architecture reconstruction by assuming that most well-designed systems follow strict architectural design rules (Cai et al., 2013). Klint et al. develop RASCAL, a domain-specific language that integrates source code analysis, transformation, and generation at the language level (Klint et al., 2009). However, the UML diagrams produced by Garzón et al. are not suitable for quality prediction, and the design rules proposed by Cai et al. take the form of special program constructs that none of the submodules use. In addition, RASCAL does not support the generation of models with a different metamodel than the analyzed code. Reis et al. discuss XIS-Reverse (Reis and da Silva, 2017), an MDRE approach that extracts data from legacy relational database schemas to create high-level system representations. However, it is limited to relational databases and excludes other types of artifacts. Cosentino et al. present a model-driven framework for reverse engineering business rules from Java applications to improve system maintainability and

adaptability (Cosentino et al., 2012). However, the framework is specific to Java and business rules, which limits its applicability to other languages and formats. Yang et al. describe C2AADL_Reverse (Yang et al., 2021), an MDRE approach that converts multitask C source code into AADL models, primarily for code verification. However, it does not use additional code analysis to improve architectural understanding. Johnson et al. propose a method for enabling Internet-based interoperation between legacy information systems with minimal changes to existing code (Johnson and Rosebrugh, 2001). This approach involves reverse engineering only the necessary parts of these systems to develop a model for interoperation. However, it does not represent the complete system, nor does it support operations beyond modifications to the existing code. Haitzer et al. propose a design science approach to align software architecture with source code to prevent erosion and drift (Haitzer et al., 2017). Their approach focuses on maintaining architectural consistency through evolutionary styles and reconstruction, in contrast to our view composition and refinement. Genfer et al. suggest relying on static code analysis to identify dependencies in microservice APIs, applying code detectors to model communication, and defining architectural metrics (Genfer and Zdun, 2021). Our approach integrates architectural models from multiple views, which improves adaptability to diverse and evolving systems. Krogmann presented SoMoX (Krogmann et al., 2010), which clusters existing classes into components using a graph-based technique based on metrics such as cohesion and coupling. Langhammer presented Extract and EJBmoX (Langhammer et al., 2016), reverse engineering tools that extract source code behavior using hybrid analysis. Walter presented PMX (Walter et al., 2017), which automatically builds the prediction model from form measurement logs from the Kieker monitoring tool (Van Hoorn et al., 2012). Grohmann et al. presented SARDE (Grohmann et al., 2021), which integrates Continuous and Self-Adaptive Resource Demand Estimation and extends the hybrid extraction approach to incrementally extract models for use in a continuous integration pipeline. However, all of these approaches are specific to a particular mapping between source code and architectural model and do not support the extraction of mixed technology software in a system.

6.2. Applies a similar technique to a different problem

Our research is closely related to view-based development methodologies. View-based modeling techniques (Brunelière et al., 2018; Cicchetti et al., 2019; Meier et al., 2020b) allow the definition of views on architectural models to manage their complexity. Query-based views such as *ModelJoin* (Burger et al., 2014), EMF views (Brunelière et al., 2015), and others (Werner et al., 2019) can be dynamically created and their view types quickly changed. Meier et al. (2020a) proposes a method for defining viewpoints and views using generic operators, while Atkinson and Tunjic (2017) present a framework for multidimensional views in projective modeling. However, these methodologies require existing architectural models that instantiate MDE-compliant metamodels (typically EMF-based), and thus do not address automated reverse engineering of architectural models from heterogeneous artifacts such as code and build files. Bork et al. present a reverse engineering approach that uses template-based parsing to facilitate model-driven development and ensure model-code synchronization (Bork et al., 2008). However, it focuses on extracting views of specific artifacts without providing a holistic system perspective. Single-underlying models (Klare et al., 2021) allow the reuse of existing models and their treatment as a single model, such as a detailed architectural model. This is typically achieved by transformation networks (Stevens, 2020) or multiary transformations (Cleve et al., 2019; Bergmann, 2021). For example, in the case of Vitruvius (Klare et al., 2021), model transformations are used to link all models into a network where changes are propagated transitively. In the latter case, more than two models are synchronized with a single model transformation. A third alternative is the merging of existing models (Brunet et al., 2006). While single-underlying models can reuse and merge existing architectural models, they rely on MDE-compliant (meta)models and do not support reverse engineering from heterogeneous artifacts. In contrast, our methodology is designed to automatically reverse engineer a comprehensive and MDE-compliant architectural model from various artifacts such as code, models, build and configuration files. It also supports predefined views of different parts of the architecture for different stakeholders.

6.3. Solves a similar problem using a similar technique

Raibulet et al. compare in depth 15 different model-driven reverse engineering approaches in their literature review and find that both these approaches and their areas of application are versatile (Raibulet et al., 2017). In this respect, MoDisco is the most related approach in a comprehensive scope. Bruneliere et al. developed MoDisco (Bruneliere et al., 2014), a generic and extensible model-driven reverse engineering approach. MoDisco provides support for Java and XML technologies, which are employed to generate model-based views of the artifacts. The discovery step of MoDisco was applied to the Java projects listed in Table 2, and the generated models were uploaded to our data set (Kirschner et al., 2024). It is, however, important to note that our approach, which specializes in reverse engineering software architecture models for quality prediction, is not fully supported by a broader approach such as MoDisco. In contrast, MoDisco is designed to extract models from Java code and JavaServer Pages, and does not offer mappings to higher levels of abstraction. Although MoDisco is extensible with technologies, it does not support the direct reuse of common concepts of a technology. Moreover, combining multiple technologies to generate a view is not supported. In their systematic literature review, Mushtaq et al. examined 56 out of 3820 papers to identify research gaps in the area of multilingual code analysis (Mushtaq et al., 2017). Of these 56 papers, MoDisco and three others come closest to our approach. Lehnert et al. present a tool (Lehnert et al., 2013) that uses UML diagrams to present different views, each representing different facets of an application at different levels of abstraction. Di Lucca et al. present WARE (Di Lucca et al., 2002), a tool that uses UML diagrams to model different aspects of an application at different levels of abstraction. Tonella et al. propose a dynamic analysis method (Tonella and Ricca, 2002) to derive the model of an application through its execution. These three works focus on the domain of web applications and have challenges in handling large and complex web applications with complex dependencies between artifacts. Furthermore, they do not provide comparable quality prediction models. Ali et al. show in their empirical study of architectural consistency (Ali et al., 2018) that current tools such as Structure 101 and Lattix support multiple programming languages and extensibility. However, they lack standard support for heterogeneous systems. Structure 101 can abstract a system into a language-independent graph for analysis, while Lattix can analyze UML and SysML models, which is potentially useful for reverse engineering heterogeneous systems. Fradet et al. present a concept of views (Fradet et al., 1999), represented as graphical representations with multiplicities, and propose a simple algorithm for verifying consistency. Despite its simplicity, the proposed technique is not suitable for all types of software architectures. The effectiveness of this method can vary greatly depending on the complexity and unique requirements of the software system. Ducasse et al. present Moose (Ducasse et al., 2005), a platform-independent environment for reverse engineering and reengineering complex software systems. However, the success of Moose depends on the specific requirements and constraints of the software system to be reengineered. Müller et al. present JQAssistant, an approach to create a uniform data source for software analysis and visualization (Müller et al., 2018). However, only dependencies on the level of methods are considered so that there is no possibility to process fine granular information, which is needed for the creation

of models for quality prediction. Sabir et al. present Src2MoF (Sabir et al., 2019), a MDRE framework that transforms Java source code into UML diagrams. This framework is specific to Java and UML, limiting its applicability to other programming languages and diagram formats. Tran et al. propose a view-based reverse engineering approach to improve the interoperability and reusability of process models in service-oriented architectures (Tran et al., 2008). This approach uses stakeholder views and a partial interpreter pattern to integrate process models. However, it lacks a comprehensive method for resolving view conflicts and does not address event and error handling in processes.

7. Conclusion

In conclusion, we presented a novel approach, known as *Retriever*, to the challenge of reverse engineering software architecture models, particularly for Web service or microservice systems. The *Retriever* approach provides comprehensive answers to the two research questions from Section 1.

To address RQ1, the *Retriever* approach demonstrates an effective method for reverse engineering software architecture from disparate artifacts across different technologies. It does this by developing a knowledge representation model for different artifacts that includes model-to-model transformations that consider technology-specific relationships and concepts. This allows the approach to handle different formats and languages and translate them into a common representation for architectural modeling.

To address RQ2, the *Retriever* approach provides a framework for integrating individual views into a unified architectural model. This is achieved through model transformation and composition techniques that map concepts between views and incorporate them into a unified model based on an existing metamodel. This results in a unified model that addresses different concerns of a software system.

The evaluation of the approach demonstrates its effectiveness in real projects of different sizes and shows promising scalability and reduction quality. It is applicable to relevant software systems with different technologies and can generate an abstraction of the system as an architectural model. The approach is scalable and transferable to larger real systems, making it suitable for industrial applications. However, it also reveals areas for improvement, particularly in the identification of components that do not directly contribute to the functions of the system domain and in the analysis of component interfaces. The accuracy of the approach is found to depend on the initial phase of the extraction transformation, which converts artifact models into architectural views. The results indicate that technology-specific extraction rules significantly influence the result.

Overall, the *Retriever* approach represents a substantial contribution to the fields of software architecture reverse engineering, model-driven software engineering, and component-based software engineering. It offers a promising solution to the challenges posed by the heterogeneity and complexity of modern software systems. However, further refinement and optimization are needed to improve its performance and accuracy, especially for larger systems, and to improve the accuracy of component and interface analysis. Similarly, future research will be devoted to a more comprehensive investigation of the effort and complexity associated with defining our extraction rules. To this end, further empirical studies could examine the reuse of these rules across applications with similar technology underpinnings, as well as the effort required to define rules that focus on recurring project-specific patterns. Future work will also explore various techniques, such as parallelization of model transformations and concurrent implementation of transformation rules, to address these issues. We currently generate models from source code and configuration files, but we plan to include test cases and documentation in future iterations (Keim et al., 2019). There may also be a focus on extending the approach to handle more complex systems and integrating it with other software engineering tools and techniques, like asynchronous message-based communication (Singh et al., 2021). Similarly, the insights derived from static code analysis regarding the behavior of system components can be enhanced by the findings of runtime log file analysis, which can be employed to provide more comprehensive support for runtime views (Walter et al., 2017). This ongoing research has great potential to advance our understanding and management of complex software systems.

CRediT authorship contribution statement

Yves R. Kirschner: Writing – review & editing, Writing – original draft, Validation, Supervision, Software, Project administration, Methodology, Investigation, Data curation, Conceptualization. **Moritz Gstür:** Writing – review & editing, Writing – original draft, Software, Methodology, Conceptualization. **Timur Sağlam:** Writing – review & editing, Writing – original draft, Investigation. **Sebastian Weber:** Writing – review & editing, Validation, Investigation, Data curation. **Anne Koziolek:** Writing – review & editing, Validation, Supervision, Resources, Project administration, Methodology, Conceptualization.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used Grammarly,[3] LanguageTool,[4] and DeepL[5] Translator and Write in order to recommend and insert replacement text, perform spelling or grammar checks and corrections, or do language translations. After using these tools/services, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.jss.2024.112277.

Data availability

The implementation and the case studies are available on GitHub. We also published a benchmark in which we transparently applied our approach to the case studies.

Dataset and Replication Package for the View-Based Retriever Approach To Reverse Engineering Software Architecture Models (Original data) (Zenodo)

References

Ali, N., Baker, S., O'Crowley, R., Herold, S., Buckley, J., 2018. Architecture consistency: State of the practice, challenges and requirements. Empir. Softw. Eng. 23, 224–258. http://dx.doi.org/10.1007/s10664-017-9515-3.

Andritsos, P., Tsaparas, P., Miller, R.J., Sevcik, K.C., 2004. LIMBO: Scalable clustering of categorical data. In: Bertino, E., Christodoulakis, S., Plexousakis, D., Christophides, V., Koubarakis, M., Böhm, K., Ferrari, E. (Eds.), EDBT'04. Vol. 2992, Springer, pp. 123–146. http://dx.doi.org/10.1007/978-3-540-24741-8_9.

Atkinson, C., Stoll, D., Bostan, P., 2010. Orthographic software modeling: A practical approach to view-based development. In: Evaluation of Novel Approaches to Software Engineering. In: Communications in Computer and Information Science, Springer, pp. 206–219. http://dx.doi.org/10.1007/978-3-642-14819-4_15.

Atkinson, C., Tunjic, C., 2017. A deep view-point language for projective modeling. Inf. Syst. 133–142. http://dx.doi.org/10.1109/edoc.2017.26.

Basili, G., Caldiera, V.R., Rombach, H.D., 1994. The goal question metric approach. Encycl. Softw. Eng..

[3] https://grammarly.com

[4] https://languagetool.org

[5] https://www.deepl.com

Becker, S., Koziolek, H., Reussner, R., 2007. Model-based performance prediction with the palladio component model. In: Cortellessa, V., Uchitel, S., Yankelevich, D. (Eds.), 6 International Workshop on Software and Performance. ACM, pp. 54–65. http://dx.doi.org/10.1145/1216993.1217006.

Becker, S., Koziolek, H., Reussner, R., 2009. The Palladio component model for model-driven performance prediction. J. Syst. Softw. 82 (1), 3–22. http://dx.doi.org/10.1016/j.jss.2008.03.066.

Bergmann, G., 2021. Controllable and decomposable multidirectional synchronizations. Softw. Syst. Model. 20 (5), 1735–1774. http://dx.doi.org/10.1007/s10270-021-00879-w.

Bettini, L., 2016. Implementing Domain-Specific Languages with Xtext and Xtend. Packt Publishing Ltd.

Bogner, J., Fritzsch, J., Wagner, S., Zimmermann, A., 2019. Microservices in industry: Insights into technologies, characteristics, and software quality. In: 2019 IEEE International Conference on Software Architecture Companion. ICSA-C, IEEE, pp. 187–195. http://dx.doi.org/10.1109/icsa-c.2019.00041.

Bork, M., Geiger, L., Schneider, C., Zündorf, A., 2008. Towards roundtrip engineering-a template-based reverse engineering approach. In: Model Driven Architecture–Foundations and Applications: 4th European Conference, ECMDA-FA 2008, Berlin, Germany, June 9-13, 2008. Proceedings 4. Springer, Springer Berlin Heidelberg, pp. 33–47. http://dx.doi.org/10.1007/978-3-540-69100-6_3.

Brunelière, H., Burger, E., Cabot, J., Wimmer, M., 2018. A feature-based survey of model view approaches. In: Wasowski, A., Paige, R.F., Haugen, Ø. (Eds.), 21th ACM/IEEE International Conference on Model Driven Engineering Languages and Systems. MODELS '18, ACM, Copenhagen, Denmark, http://dx.doi.org/10.1145/3239372.3242895.

Bruneliere, H., Cabot, J., Dupé, G., Madiot, F., 2014. Modisco: A model driven reverse engineering framework. Inf. Softw. Technol. 56 (8), 1012–1032. http://dx.doi.org/10.1016/j.infsof.2014.04.007.

Brunelière, H., Perez, J.G., Wimmer, M., Cabot, J., 2015. EMF views: A view mechanism for integrating heterogeneous models. In: Johannesson, P., Lee, M.-L., Liddle, S.W., Opdahl, A.L., López, O.P. (Eds.), 34th International Conference on Conceptual Modeling. ER 2015, Vol. 9381, Springer International Publishing, Stockholm, Sweden, pp. 317–325. http://dx.doi.org/10.1007/978-3-319-25264-3_23.

Brunet, G., Chechik, M., Easterbrook, S., Nejati, S., Niu, N., Sabetzadeh, M., 2006. A manifesto for model merging. In: 2006 International Workshop on Global Integrated Model Management. GaMMa '06, ACM, Shanghai, China, pp. 5–12. http://dx.doi.org/10.1145/1138304.1138307.

Burger, E.J., 2013. Flexible views for view-based model-driven development. In: 18th International Doctoral Symposium on Components and Architecture. WCOP '13, ACM, Vancouver, British Columbia, Canada, pp. 25–30. http://dx.doi.org/10.1145/2465498.2465501.

Burger, E., Henß, J., Küster, M., Kruse, S., Happe, L., 2014. View-based model-driven software development with ModelJoin. Softw. Syst. Model. 15 (2), 473–496. http://dx.doi.org/10.1007/s10270-014-0413-5.

Cai, Y., Wang, H., Wong, S., Wang, L., 2013. Leveraging design rules to improve software architecture recovery. In: Kruchten, P., Koziolek, A., Nord, R.L. (Eds.), 9 International ACM Sigsoft Conference on Quality of Software Architectures. QoSA '13, ACM, Vancouver, British Columbia, Canada, pp. 133–142. http://dx.doi.org/10.1145/2465478.2465480.

Canfora, G., Di Penta, M., 2007. New frontiers of reverse engineering. In: Briand, L.C., Wolf, A.L. (Eds.), 2007 Future of Software Engineering. FOSE '07, IEEE Computer Society, pp. 326–341. http://dx.doi.org/10.1109/fose.2007.15.

Canfora, G., Di Penta, M., Cerulo, L., 2011. Achievements and challenges in software reverse engineering. Commun. ACM 54 (4), 142–151. http://dx.doi.org/10.1145/1924421.1924451.

Chu, W.C., Lu, C.-W., Chang, C.-H., Chung, Y.-C., Liu, X., Yang, H., 2002. Reverse engineering. In: Handbook of Software Engineering and Knowledge Engineering: Volume II: Emerging Technologies. World Scientific, pp. 447–466.

Cicchetti, A., Ciccozzi, F., Pierantonio, A., 2019. Multi-view approaches for software and system modelling: a systematic literature review. Softw. Syst. Model. 18 (6), 3207–3233. http://dx.doi.org/10.1007/s10270-018-00713-w.

Cleve, A., Kindler, E., Stevens, P., Zaytsev, V., 2019. Multidirectional transformations and synchronisations (dagstuhl seminar 18491). Dagstuhl Rep. 8 (12), 1–48. http://dx.doi.org/10.4230/dagrep.8.12.1.

Cosentino, V., Cabot, J., Albert, P., Bauquel, P., Perronnet, J., 2012. A model driven reverse engineering framework for extracting business rules out of a java application. In: International Workshop on Rules and Rule Markup Languages for the Semantic Web. Springer, Springer Berlin Heidelberg, pp. 17–31. http://dx.doi.org/10.1007/978-3-642-32689-9_3.

Di Lucca, G., Fasolino, A., Pace, F., Tramontana, P., De Carlini, U., 2002. WARE: a tool for the reverse engineering of Web applications. In: Sixth European Conference on Software Maintenance and Reengineering. IEEE, pp. 241–250. http://dx.doi.org/10.1109/csmr.2002.995811.

Ducasse, S., Gîrba, T., Nierstrasz, O., 2005. Moose: An agile reengineering environment. In: Wermelinger, M., Gall, H. (Eds.), 10th European Software Engineering Conference Jointly with 13th International Symposium on Foundations of Software Engineering. ACM, Lisbon, Portugal, http://dx.doi.org/10.1145/1081706.1081723.

Favre, J.-M., 2005. Foundations of model (driven) (reverse) engineering : Models – episode I: Stories of the fidus papyrus and of the solarus. In: Language Engineering for Model-Driven Software Development. In: Dagstuhl Seminar Proceedings (DagSemProc).

fortiss, 2023. CCE trends. URL https://cce.fortiss.org/trends.

Fradet, P., Le Métayer, D., Périn, M., 1999. Consistency checking for multiple view software architectures. SIGSOFT Softw. Eng. Notes 24 (6), 410–428. http://dx.doi.org/10.1145/318774.319258.

Garcia, J., Ivkovic, I., Medvidovic, N., 2013. A comparative analysis of software architecture recovery techniques. In: ASE'13. IEEE, pp. 486–496. http://dx.doi.org/10.1109/ase.2013.6693106.

Garcia, J., Popescu, D., Mattmann, C., Medvidovic, N., Cai, Y., 2011. Enhancing architectural recovery using concerns. In: Alexander, P., Pasareanu, C.S., Hosking, J.G. (Eds.), ASE'11. IEEE, pp. 552–555. http://dx.doi.org/10.1109/ase.2011.6100123.

Garzón, M.A., Lethbridge, T.C., Aljamaan, H.I., Badreddin, O., 2014. Reverse engineering of object-oriented code into Umple using an incremental and rule-based approach. In: Ng, J., Li, J., Wong, K. (Eds.), CASCON'14. IBM / ACM, pp. 91–105.

Genfer, P., Zdun, U., 2021. Identifying domain-based cyclic dependencies in microservice APIs using source code detectors. In: Biffl, S., Navarro, E., Löwe, W., Sirjani, M., Mirandola, R., Weyns, D. (Eds.), Software Architecture. In: Lecture Notes in Computer Science Ser., vol. 12857, Springer International Publishing, Cham, pp. 207–222, Description based on publisher supplied metadata and other sources.

Goldschmidt, t., Becker, S., Burger, E., 2012. Towards a tool-oriented taxonomy of view-based modelling. In: Sinz, E.J., Schürr, A. (Eds.), Modellierung 2012. In: GI-Edition – Lecture Notes in Informatics (LNI), vol. 201, Gesellschaft für Informatik e.V. (GI), Bamberg, pp. 59–74.

Götz, S., Tichy, M., Groner, R., 2021. Claimed advantages and disadvantages of (dedicated) model transformation languages: a systematic literature review. Softw. Syst. Model. 20, 45–46.

Grohmann, J., Eismann, S., Bauer, A., Spinner, S., Blum, J., Herbst, N., Kounev, S., 2021. SARDE: A framework for continuous and self-adaptive resource demand estimation. ACM Trans. Auton. Adapt. Syst. 15 (2), http://dx.doi.org/10.1145/3463369.

Gruhn, V., Striemer, R., 2018. The Essence of Software Engineering. Springer Nature, http://dx.doi.org/10.1007/978-3-319-73897-0.

Haitzer, T., Navarro, E., Zdun, U., 2017. Reconciling software architecture and source code in support of software evolution. J. Syst. Softw. 123, 119–144. http://dx.doi.org/10.1016/j.jss.2016.10.012, URL https://www.sciencedirect.com/science/article/pii/S0164121216302114.

Hasselbring, W., 2002. Component-based software engineering. In: Handbook of Software Engineering and Knowledge Engineering: Volume II: Emerging Technologies. World Scientific, pp. 289–305.

Hebig, R., Seidl, C., Berger, t., Pedersen, J.K., Wąsowski, A., 2018. Model transformation languages under a magnifying glass: a controlled experiment with Xtend, ATL, and QVT. In: Leavens, G.T., Garcia, A., Pasareanu, C.S. (Eds.), Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ACM, http://dx.doi.org/10.1145/3236024.3236046.

ISO Central Secretary, 2022. Software, Systems and Enterprise – Architecture Description. Standard, International Organization for Standardization, http://dx.doi.org/10.3403/30373968.

Johnson, M., Rosebrugh, R., 2001. Engineering legacy information systems for internet based interoperation. In: Proceedings IEEE International Conference on Software Maintenance. ICSM 2001. IEEE, IEEE Comput. Soc, pp. 32–39. http://dx.doi.org/10.1109/icsm.2001.972709.

Jordan, S., Linsbauer, L., Schaefer, I., 2022. AutoArx: Digital twins of living architectures. In: Gerostathopoulos, I., Lewis, G.A., Batista, T.V., Bures, T. (Eds.), European Conference on Software Architecture. Vol. 13444, Springer, Springer Science+Business Media, pp. 205–212. http://dx.doi.org/10.1007/978-3-031-16697-6_15.

Jouault, F., Allilaire, F., Bézivin, J., Kurtev, I., 2008. ATL: A model transformation tool. Sci. Comput. Program. 72 (1–2), 31–39. http://dx.doi.org/10.1016/j.scico.2007.08.002.

Kazman, R., Bass, L., Abowd, G., Webb, M., 1994. SAAM: a method for analyzing the properties of software architectures. In: Proceedings of 16th International Conference on Software Engineering. IEEE Comput. Soc. Press, pp. 81–90. http://dx.doi.org/10.1109/icse.1994.296768.

Kazman, R., Woods, S., Carriere, S., 1998. Requirements for integrating software architecture and reengineering models: CORUM II. In: Proceedings Fifth Working Conference on Reverse Engineering. IEEE Comput. Soc, pp. 154–163. http://dx.doi.org/10.1109/wcre.1998.723185.

Keim, J., Schneider, Y., Koziolek, A., 2019. Towards consistency analysis between formal and informal software architecture artefacts. In: Medvidovic, N., Mirakhorli, M., Malek, S., Garcia, J., Cai, Y., Kazman, R., Xiao, L. (Eds.), 2019 IEEE/ACM 2nd International Workshop on Establishing the Community-Wide Infrastructure for Architecture-Based Software Engineering. ECASE, IEEE, IEEE / ACM, pp. 6–12. http://dx.doi.org/10.1109/ecase.2019.00010.

Kent, S., 2002. Model driven engineering. In: Butler, M.J., Petre, L., Sere, K. (Eds.), Integrated Formal Methods. Vol. 2335, Springer Berlin Heidelberg, pp. 286–298. http://dx.doi.org/10.1007/3-540-47884-1_16.

Kirschner, Y.R., Bossert, F., Gstür, M., Koziolek, A., 2023. Retriever repository. URL https://github.com/PalladioSimulator/Palladio-ReverseEngineering-Retriever.

Kirschner, Y.R., Keim, J., Peter, N., Koziolek, A., 2023a. Automated reverse engineering of the technology-induced software system structure. In: Tekinerdogan, B., Trubiani, C., Tibermacine, C., Scandurra, P., Cuesta, C.E. (Eds.), Software Architecture, first ed. In: Lecture Notes in Computer Science, vol. 14212, Springer Nature Switzerland, Cham, pp. 283–291. http://dx.doi.org/10.1007/978-3-031-42592-9_19.

Kirschner, Y., Moritz, G., Sağlam, T., Weber, S., Koziolek, A., 2024. Dataset and replication package for the view-based retriever approach to reverse engineering software architecture models. http://dx.doi.org/10.5281/zenodo.11144303.

Kirschner, Y.R., Walter, M., Bossert, F., Heinrich, R., Koziolek, A., 2023b. Automatic derivation of vulnerability models for software architectures. In: 2023 IEEE 20th International Conference on Software Architecture Companion. ICSA-C, IEEE, pp. 276–283. http://dx.doi.org/10.1109/icsa-c57050.2023.00065.

Klare, H., Kramer, M.E., Langhammer, M., Werle, D., Burger, E., Reussner, R., 2021. Enabling consistency in view-based system development — the Vitruvius approach. J. Syst. Softw. 171, 110815. http://dx.doi.org/10.1016/j.jss.2020.110815.

Klint, P., van der Storm, T., Vinju, J., 2009. RASCAL: A domain specific language for source code analysis and manipulation. In: ICSME'09. IEEE, pp. 168–177. http://dx.doi.org/10.1109/scam.2009.28.

Krogmann, K., Kuperberg, M., Reussner, R., 2010. Using genetic search for reverse engineering of parametric behavior models for performance prediction. IEEE Trans. Softw. Eng. 36 (6), 865–877. http://dx.doi.org/10.1109/tse.2010.69.

Kruchten, P., 1995. The 4+1 view model of architecture. IEEE Softw. 12, 42–50. http://dx.doi.org/10.1109/52.469759.

Kruchten, P., 2004. The Rational Unified Process: An Introduction, third ed. In: The @Addison-Wesley Object Technology Series, Addison-Wesley Professional, Upper Saddler River, NJ [u.a.], Literaturverz. S. 291 - 297.

Kurtev, I., 2008. State of the art of QVT: A model transformation language standard. In: AGTIVE 2007: Applications of Graph Transformations with Industrial Relevance: Third International Symposium. Springer, Springer Berlin Heidelberg, pp. 377–393. http://dx.doi.org/10.1007/978-3-540-89020-1_26.

Langhammer, M., Shahbazian, A., Medvidovic, N., Reussner, R.H., 2016. Automated extraction of rich software models from limited system information. In: 2016 13th Working IEEE/IFIP Conference on Software Architecture. WICSA, IEEE, pp. 99–108. http://dx.doi.org/10.1109/wicsa.2016.35.

Lehnert, S., Farooq, Q.-u.-a., Riebisch, M., 2013. Rule-based impact analysis for heterogeneous software artifacts. In: Cleve, A., Ricca, F., Cerioli, M. (Eds.), 2013 17th European Conference on Software Maintenance and Reengineering. IEEE, pp. 209–218. http://dx.doi.org/10.1109/csmr.2013.30.

Lungu, M., 2008. Towards reverse engineering software ecosystems. In: 2008 IEEE International Conference on Software Maintenance. IEEE, pp. 428–431. http://dx.doi.org/10.1109/icsm.2008.4658096.

Meier, J., Kateule, R., Winter, A., 2020a. Operator-based viewpoint definition. In: Hammoudi, S., Pires, L.F., Selic, B. (Eds.), 8th International Conference on Model-Driven Engineering and Software Development - MODELSWARD. INSTICC, SciTePress, pp. 401–408. http://dx.doi.org/10.5220/0008977404010408.

Meier, J., Werner, C., Klare, H., Tunjic, C., Aßmann, U., Atkinson, C., Burger, E., Reussner, R., Winter, A., 2020b. Classifying approaches for constructing single underlying models. In: Model-Driven Engineering and Software Development. Springer International Publishing, pp. 350–375. http://dx.doi.org/10.1007/978-3-030-37873-8_15.

Mens, T., Van Gorp, P., 2006. A taxonomy of model transformation. Electron. Notes Theor. Comput. Sci. 152, 125–142. http://dx.doi.org/10.1016/j.entcs.2005.10.021.

Mitchell, B.S., Mancoridis, S., 2006. On the automatic modularization of software systems using the bunch tool. IEEE Trans. Softw. Eng. 32 (3), 193–208. http://dx.doi.org/10.1109/tse.2006.31.

Müller, R., Mahler, D., Hunger, M., Nerche, J., Harrer, M., 2018. Towards an open source stack to create a unified data source for software analysis and visualization. In: VISSOFT'18. IEEE, pp. 107–111. http://dx.doi.org/10.1109/vissoft.2018.00019.

Murer, S., Bonati, B., Furrer, F.J., 2011. Managed Evolution: A Strategy for Very Large Information Systems, first ed. Springer Berlin Heidelberg.

Mushtaq, Z., Rasool, G., Shehzad, B., 2017. Multilingual source code analysis: A systematic literature review. IEEE Access 5, 11307–11336. http://dx.doi.org/10.1109/access.2017.2710421.

Object Management Group, 2017. Unified Modeling Language (UML), v2.5.1. URL https://www.omg.org/spec/UML/2.5.1.

Pfeiffer, R.-H., 2020. What constitutes software? An empirical, descriptive study of artifacts. In: Kim, S., Gousios, G., Nadi, S., Hejderup, J. (Eds.), 17th International Conference on Mining Software Repositories. MSR '20, ACM, Seoul, Republic of Korea, pp. 481–491. http://dx.doi.org/10.1145/3379597.3387442.

Raibulet, C., Fontana, F.A., Zanoni, M., 2017. Model-driven reverse engineering approaches: A systematic literature review. IEEE Access 5, 14516–14542. http://dx.doi.org/10.1109/access.2017.2733518.

Reis, A., da Silva, A.R., 2017. XIS-Reverse: a model-driven reverse engineering approach for legacy information systems. In: International Conference on Model-Driven Engineering and Software Development. Vol. 2, SCITEPRESS, SCITEPRESS - Science and Technology Publications, pp. 196–207. http://dx.doi.org/10.5220/0006271501960207.

Reussner, R., Becker, S., Burger, E., Happe, J., Hauck, M., Koziolek, A., Koziolek, H., Krogmann, K., Kuperberg, M., 2011. The Palladio Component Model. Tech. Rep. 14, Karlsruher Institut für Technologie (KIT), http://dx.doi.org/10.5445/IR/1000022503.

Reussner, R.H., Becker, S., Happe, J., Heinrich, R., Koziolek, A., Koziolek, H., Kramer, M., Krogmann, K. (Eds.), 2016. Modeling and Simulating Software Architectures – The Palladio Approach. MIT Press.

Roques, A., 2023. PlantUML. URL https://plantuml.com.

Rugaber, S., Stirewalt, K., 2004. Model-driven reverse engineering. IEEE Softw. 21 (4), 45–53. http://dx.doi.org/10.1109/ms.2004.23.

Runeson, P., Höst, M., 2008. Guidelines for conducting and reporting case study research in software engineering. Empir. Softw. Eng. 14 (2), 131–164. http://dx.doi.org/10.1007/s10664-008-9102-8.pdf.

Sabir, U., Azam, F., Haq, S.U., Anwar, M.W., Butt, W.H., Amjad, A., 2019. A model driven reverse engineering framework for generating high level UML models from Java source code. IEEE Access 7, 158931–158950. http://dx.doi.org/10.1109/access.2019.2950884.

Selfridge, P., Waters, R., Chikofsky, E., 1993. Challenges to the field of reverse engineering. In: Proceedings Working Conference on Reverse Engineering. IEEE Comput. Soc. Press, pp. 144–150. http://dx.doi.org/10.1109/wcre.1993.287770.

Singh, S., Kirschner, Y.R., Koziolek, A., 2021. Towards extraction of message-based communication in mixed-technology architectures for performance model. In: Bourcier, J., Jiang, Z.M.J., Bezemer, C.-P., Cortellessa, V., Pompeo, D.D., Varbanescu, A.L. (Eds.), Companion of the ACM/SPEC International Conference on Performance Engineering. ACM, pp. 133–138. http://dx.doi.org/10.1145/3447545.3451201.

Stachowiak, H., 1973. Allgemeine Modelltheorie. Springer-Verlag, Wien.

Stack Overflow, 2023. Stack overflow developer survey 2023. URL https://survey.stackoverflow.co/2023.

Steinberg, D., Budinsky, F., Merks, E., Paternostro, M., 2008. EMF: Eclipse Modeling Framework. Pearson Education.

Stevens, P., 2020. Maintaining consistency in networks of models: bidirectional transformations in the large. Softw. Syst. Model. 19, 39–65. http://dx.doi.org/10.1007/s10270-019-00736-x.

Szyperski, C., Gruntz, D., Murer, S., 2002. Component Software: Beyond Object-Oriented Programming. Pearson Education.

Tonella, P., Ricca, F., 2002. Dynamic model extraction and statistical analysis of Web applications. In: Proceedings. Fourth International Workshop on Web Site Evolution. In: WSE-02, IEEE Comput. Soc, pp. 43–52. http://dx.doi.org/10.1109/wse.2002.1134088.

Tran, H., Zdun, U., Dustdar, S., 2008. View-based reverse engineering approach for enhancing model interoperability and reusability in process-driven SOAs. In: International Conference on Software Reuse. Springer, Springer Berlin Heidelberg, pp. 233–244. http://dx.doi.org/10.1007/978-3-540-68073-4_23.

Tzerpos, V., Holt, R.C., 2000. ACCD: an algorithm for comprehension-driven clustering. In: WCRE'00. IEEE, pp. 258–267. http://dx.doi.org/10.1109/wcre.2000.891477.

Van Hoorn, A., Waller, J., Hasselbring, W., 2012. Kieker: A framework for application performance monitoring and dynamic software analysis. In: Kaeli, D.R., Rolia, J., John, L.K., Krishnamurthy, D. (Eds.), 3 ACM/SPEC International Conference on Performance Engineering. ACM, pp. 247–248. http://dx.doi.org/10.1145/2188286.2188326.

Van Rijsbergen, C., Van Rijsbergen, C., 1979. Information Retrieval. Butterworths.

Völter, M., Stahl, t., Bettin, J., Haase, A., Helsen, S., 2013. Model-Driven Software Development: Technology, Engineering, Management. John Wiley & Sons.

Walter, J., Stier, C., Koziolek, H., Kounev, S., 2017. An expandable extraction framework for architectural performance models. In: Binder, W., Cortellessa, V., Koziolek, A., Smirni, E., Poess, M. (Eds.), 8 ACM/SPEC on International Conference on Performance Engineering Companion. ACM, http://dx.doi.org/10.1145/3053600.3053634.

Werner, C., Wimmer, M., Assmann, U., 2019. A generic language for query and viewtype generation by-example. In: Burgueño, L., Pretschner, A., Voss, S., Chaudron, M., Kienzle, J., Völter, M., Gérard, S., Zahedi, M., Bousse, E., Rensink, A., Polack, F., Engels, G., Kappel, G. (Eds.), 2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion. IEEE, pp. 379–386. http://dx.doi.org/10.1109/models-c.2019.00059.

Wittler, J.W., Sağlam, T., Kühn, t., 2023. Evaluating model differencing for the consistency preservation of state-based views. J. Object Technol. 22 (2), 2:1. http://dx.doi.org/10.5381/jot.2023.22.2.a4, The 19th European Conference on Modelling Foundations and Applications (ECMFA 2023).

Yang, Z., Qiu, Z., Zhou, Y., Huang, Z., Bodeveix, J.-P., Filali, M., 2021. C2AADL_Reverse: A model-driven reverse engineering approach to development and verification of safety-critical software. J. Syst. Archit. 118, 102202. http://dx.doi.org/10.1016/j.sysarc.2021.102202.

Yves Kirschner has served as a doctoral researcher in the Modelling for Continuous Software Engineering (MCSE) research group at the Karlsruhe Institute of Technology (KIT) since 2018. His research interests lie in model-based reverse engineering for software systems, with a particular focus on the extraction of automatic quality prediction models. The objective of this research is to develop a method that enables informed trade-off decisions to be made in architecture decisions while reducing the additional workload for developers.

Moritz Gstür has been pursuing a Master's degree in Computer Science at the Karlsruhe Institute of Technology (KIT) since 2021. His Bachelor's thesis was also conducted at KIT, where he compared various reverse engineering approaches for software architectures. His studies are concentrated on the selected master's program, "Software Engineering". As part of this endeavor, he developed the original generic framework for model-based composition and rule-based refinement.

Timur Sağlam has been a doctoral researcher in the Dependability of Software-Intensive Systems (DSiS) research group at the Karlsruhe Institute of Technology (KIT) since 2020. His research is focused on the detection of software plagiarism and collusion, with a particular emphasis on the detection of obfuscation attacks on structure-based plagiarism detectors. He is the principal developer for JPlag, an open-source plagiarism detector utilized by numerous universities worldwide. Moreover, he is engaged in the advancement of the Vitruvius framework for consistent, view-based systems engineering.

Sebastian Weber has served as a doctoral researcher at the Research Center for Information Technology (FZI) in the Software Engineering (SE) research department since 2022. His work concerns the automated extraction of architectural performance models from measurement data. His research interests lie in the modeling and simulation of hardware and software at different levels of abstraction and their coupling in system architectures. His research is focused on the integration of hardware simulations into a higher-level architectural framework and the orchestration of these simulations at a higher architectural level.

Anne Koziolek has been a full professor of software engineering at the Karlsruhe Institute of Technology (KIT) since 2019. She is the chair of the research group Modelling for Continuous Software Engineering (MCSE) at the Institute for Security and Reliability in Information Technology (KASTEL). The research group's focus is on the initial stages and activities involved in the development of software-intensive technical systems. In this context, she integrates model-based software engineering with development processes that facilitate rapid and agile feedback cycles.